



NON-TERRESTERIAL GEOSPATIAL ENGINEERING REPORT

ENGINEERING REPORT

DRAFT

Submission Date: 2029-12-31

Approval Date: 2029-12-31

Publication Date: 2029-12-31

Editor: Sina Taghavikish, Rob Smith

Notice: This document is not an OGC Standard. This document is an OGC Public Engineering Report created as a deliverable in an OGC Interoperability Initiative and is *not an official position* of the OGC membership. It is distributed for review and comment. It is subject to change without notice and may not be referred to as an OGC Standard.

Further, any OGC Engineering Report should not be referenced as required or mandatory technology in procurements. However, the discussions in this document could very well lead to the definition of an OGC Standard.

License Agreement

Use of this document is subject to the license agreement at <https://www.ogc.org/license>

Copyright notice

Copyright © 2023 Open Geospatial Consortium

To obtain additional rights of use, visit <https://www.ogc.org/legal>

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

I. EXECUTIVE SUMMARY	vi
II. KEYWORDS	vii
III. SUBMITTERS	vii
1. INTRODUCTION	9
2. ISO 19111 EXTENSIONS	12
2.1. Abstract model overview	12
2.2. Additions to ISO 19111	20
2.3. Changes to ISO 19111	22
2.4. Example using DART data	24
3. EXTENSION OF GEOPOSE TO NON-TERRESTRIAL USE CASES	26
3.1. GeoPose Conceptual Framework	26
3.2. GeoPose Architecture V 1.0	27
3.3. GeoPose Suggestions from TB-18, Public Review, and Implementation	27
3.4. Implementing Selected Suggestions	29
3.5. Issues of locating terrestrial objects	31
3.6. An examination of experiments in recording the location of terrestrial objects	32
3.7. Road Network Use Cases	32
3.8. Future Work	46
ANNEX A (INFORMATIVE) ABBREVIATIONS/ACRONYMS	50
ANNEX B (INFORMATIVE) XML SCHEMA	52
B.1. Examples	56
ANNEX C (INFORMATIVE) GEOAPI INTERFACES	60
ANNEX D (INFORMATIVE) SYNCHRONIZED PREVIZUALIZATION	63
D.1. Example 17:15-17:21, 25th April 2023	64
D.2. Example 12:02-12:09, 26th April 2023	72
D.3. Example 13:08-13:14, 26th April 2023	78
BIBLIOGRAPHY	87

LIST OF TABLES

Table 1 – Reifications of ISO 19111 abstract model	9
Table 2 – Implementations of reifications of ISO 19111 abstract model	9
Table 3 – InertialCRS properties	21
Table 4 – InertialReferenceFrame properties	21
Table 5 – Nokia 5 Synchronization Metrics	41
Table 6 – Moto G30 Synchronization Metrics	41

LIST OF FIGURES

Figure 1	13
Figure 2 – Surface with local Cartesian CS in each cell	14
Figure 3 – Surface with local ellipsoidal CS in each cell	15
Figure 4 – More complex coordinate system covering larger area	17
Figure 5 – Relative errors of distances around 0°E 45°N computed with different methods	18
Figure 6	23
Figure 7	26
Figure 8	26
Figure 9	28
Figure 10 – StreetDrone Vehicle Operated By Ordnance Survey	34
Figure 11 – Roadsign A-Frame, Traffic Cone & Plastic Bottle	35
Figure 12 – Lines of Sight With Target Location Zones	36
Figure 13 – Vehicle Tracking Previzualization	38
Figure 14 – Synchronized Data Previzualization	40
Figure 15 – Target Zones To Detect Wrong Way Traffic	43
Figure 16 – Unexpected U-Turn Detection	45
Figure 17 – Identified Locations For Litter Monitoring Use Case	47
Figure B.1 – XML schema proposal for standardization by a SWG	52
Figure B.2 – Definition of Didymos inertial CRS in extended GML	56
Figure C.1 – Celestial Body interface proposal for the GeoAPI Standard (Java version)	60
Figure C.2 – Inertial Reference Frame interface proposal for the GeoAPI Standard (Java version)	60
Figure C.3 – Inertial CRS interface proposal for the GeoAPI Standard (Java version)	61
Figure C.4 – Minkowski CS interface proposal for the GeoAPI Standard (Java version)	61
Figure D.1 – Synchronized Previzualization - Moto G30, 17:15-17:21 25 April, Pass 1	64
Figure D.2 – Synchronized Previzualization - Moto G30, 17:15-17:21 25 April, Pass 2	65
Figure D.3 – Synchronized Previzualization - Moto G30, 17:15-17:21 25 April, Pass 3	66

Figure D.4 — Synchronized Previzualization - Moto G30, 17:15-17:21 25 April, Pass 4	67
Figure D.5 — Synchronized Previzualization - Nokia 5, 17:15-17:21 25 April, Pass 1	68
Figure D.6 — Synchronized Previzualization - Nokia 5, 17:15-17:21 25 April, Pass 2	69
Figure D.7 — Synchronized Previzualization - Nokia 5, 17:15-17:21 25 April, Pass 3	70
Figure D.8 — Synchronized Previzualization - Nokia 5, 17:15-17:21 25 April, Pass 4	71
Figure D.9 — Synchronized Previzualization - Moto G30, 12:02-12:09 26 April, Pass 1	72
Figure D.10 — Synchronized Previzualization - Moto G30, 12:02-12:09 26 April, Pass 2	73
Figure D.11 — Synchronized Previzualization - Nokia 5, 12:02-12:09 26 April, Pass 1	74
Figure D.12 — Synchronized Previzualization - Nokia 5, 12:02-12:09 26 April, Pass 2	75
Figure D.13 — Synchronized Previzualization - Nokia 5, 12:02-12:09 26 April, Pass 3	76
Figure D.14 — Synchronized Previzualization - Nokia 5, 12:02-12:09 26 April, Pass 4	77
Figure D.15 — Synchronized Previzualization - Moto G30, 13:08-13:14 26 April, Pass 1	78
Figure D.16 — Synchronized Previzualization - Moto G30, 13:08-13:14 26 April, Pass 2	79
Figure D.17 — Synchronized Previzualization - Moto G30, 13:08-13:14 26 April, Pass 3	80
Figure D.18 — Synchronized Previzualization - Moto G30, 13:08-13:14 26 April, Pass 4	81
Figure D.19 — Synchronized Previzualization - Nokia 5, 13:08-13:14 26 April, Pass 1	82
Figure D.20 — Synchronized Previzualization - Nokia 5, 13:08-13:14 26 April, Pass 2	83
Figure D.21 — Synchronized Previzualization - Nokia 5, 13:08-13:14 26 April, Pass 3	84
Figure D.22 — Synchronized Previzualization - Nokia 5, 13:08-13:14 26 April, Pass 4	85



EXECUTIVE SUMMARY

This OGC Testbed 19 Engineering Report (ER) provides UML diagrams, XSD schemas and Java/Python interfaces formalizing the concepts introduced in OGC Testbed 18 Engineering Reports. Testbed 18 assessed the applicability of existing OGC/ISO Standards for describing objects in orbit of any celestial body or in free flight in our solar system. The Testbed 18 had the following limitations:

- The proposed extensions to OGC/ISO Standards were in the form of discussions.
- The use case scenario was simplistic for making the demo easy to implement.
- The absence of implementation for proposed extensions forced the use of workarounds.

This Testbed 19 addresses above shortcomings as below:

- Proposed extensions to OGC/ISO Standards are formalized in machine-readable formats (XSD or Java/Python interfaces).
- The use case scenario is based on real data of a spacecraft mission (NASA's DART).
- The demo is implemented as it would be if the extensions were accepted by relevant OGC working groups.

The extensions to ISO 19111 constitute the first chapter of this ER. It is followed by a second chapter about adapting GeoPose to the spatial context.

In addition, Testbed-18 identified operational requirements for two geotagged video uses cases in Engineering Report D020 from discussions with UK national road network authorities. Existing datasets could be enriched by synchronizing geotagged video and sensor data from multiple devices and aggregating information over time. This leverages GeoPose and Moving Features by integration with WebVMT in a web browser and builds on results from Testbeds 16 and 17.

In April 2023, concurrent geotagged video and sensor data were captured at Ordnance Survey's headquarters in Southampton, UK. This collaborative effort included body-worn video devices, smartphones, a drone and a StreetDrone autonomous vehicle, and was designed to gather data suitable for several use cases:

- **Wrong way traffic** to automate detection and location of wrong way vehicles using traffic cameras.
- **Litter monitoring** to identify, locate and monitor roadside debris from fleet dashcam footage.
- **Taxi hailing** to locate taxis and kerbside passengers from video footage.

Testbed-19 addresses the wrong way traffic use case by demonstrating how roadside video footage can be synchronized with traffic flow data to quickly identify and locate vehicles travelling in the opposite direction to the traffic flow. This is an important operational issue for organisations that are responsible for public safety on the road network.



KEYWORDS

The following are keywords to be used by search engines and document catalogues.

DART, AIDA, testbed, OS Bubble



SUBMITTERS

All questions regarding this document should be directed to the editor or the contributors:

NAME	ORGANIZATION	ROLE
Rob Smith	Away Team	Contributor
Sina Taghavikish	Open Geospatial Consortium	Editor
Charles Heazel	Heazel Tech	Contributor
Jérôme Jacovella-St-Louis	Ecere	Contributor
Carl Stephen Smyth	Open Site Plan	Contributor
Martin Desruisseaux	Geomatys	Editor



1

INTRODUCTION

A Coordinate Reference System (CRS) is a framework used to precisely measure locations in space and time as coordinates. The ISO 19111 Standard, also available as OGC Topic 2, provides the abstract model for this framework. While this framework was initially designed for coordinates on the surface of Earth, it nevertheless provides structures such as EngineeringCRS that are applicable to vessels, not necessarily close to Earth surface. Those structures are already capable to provide *some* descriptions of CRS in space, and OGC 22-038r2 proposed extensions to those structures for more complete and accurate descriptions.

ISO 19111 is an abstract model defined by UML diagrams. This abstract model is expressed in different machine-readable formats by separated projects or Standards. Those projects or Standards can be seen as “implementations” of ISO 19111, but not necessarily in the sense of object-oriented programming languages. To distinguish them, this Engineering Report uses the Reification term. Some reifications cover only a subset of ISO 19111 for simplicity reasons.

Table 1 — Reifications of ISO 19111 abstract model

STANDARD OR PRODUCT	REIFICATION OF	COVER ALL THE UML?
EPSG geodetic dataset	ISO 19111:2019	No
ISO 19162:2019 – WKT 2	ISO 19111:2019	No
ISO 19136:2007 – GML	ISO 19111:2007	Yes
OGC 09-083 – GeoAPI 3.0	ISO 19111:2007	Yes
PROJ-JSON (community)	ISO 19111:2019	To be verified

In above list, GeoAPI is still abstract on an object-oriented programming point of view. “Real” implementations are provided by external projects. For example:

Table 2 — Implementations of reifications of ISO 19111 abstract model

EXTERNAL PROJECT	IMPLEMENTATION OF	COVER ALL INTERFACES?
Apache SIS	OGC 09-083 – GeoAPI 3.0.2	Yes
PROJ-JNI	OGC 09-083 – GeoAPI 3.0.2	No

This Engineering Report (ER) extends the following Standards with programmatic definitions of concepts introduced in TestBed 18:

- ISO 19111, extended with UML diagrams.
- ISO 19136, extended with XSD schemas.
- GeoAPI 3.0, extended with Java and Python interfaces.
- GeoPose, extended with **TODO**.



2

ISO 19111 EXTENSIONS

This chapter describes the ISO 19111 extensions experimented in this test bed. The same extensions are expressed in three different ways:

- As Unified Modeling Language (UML) diagrams in this engineering report.
- As an XML schema extending the Geographic Markup Language (GML).
- As Java interfaces in a branch of OGC GeoAPI.

Each of those deliverable can be assigned to an OGC working group for discussion or standardization:

- The CRS SWG can evaluate the UML diagrams for eventual incorporation in OGC Topic 2 and ISO 19111.
- The planetary DWG can evaluate the XML schema for eventual publication on <http://schemas.opengis.net/gsp/>.
- The GeoAPI SWG can evaluate the new interfaces for eventual incorporation in the GeoAPI main module.

Those extensions were initially presented in OGC 22-038r2 as informal discussions. This Testbed goes one step further by providing actual reifications and implementations of those discussions. The GML reification is presented in Annex B. The GeoAPI reification is presented in Annex C, together with an implementation based on Apache SIS. Examples are provided in Annex B.1 for the XML part.

2.1. Abstract model overview

In the following UML, the blue box contains some classes and properties defined in ISO 19111:2019. For brevity reasons, only a small subset of the classes and properties is shown. The classes and properties shown outside the blue box are additions from this Engineering report.

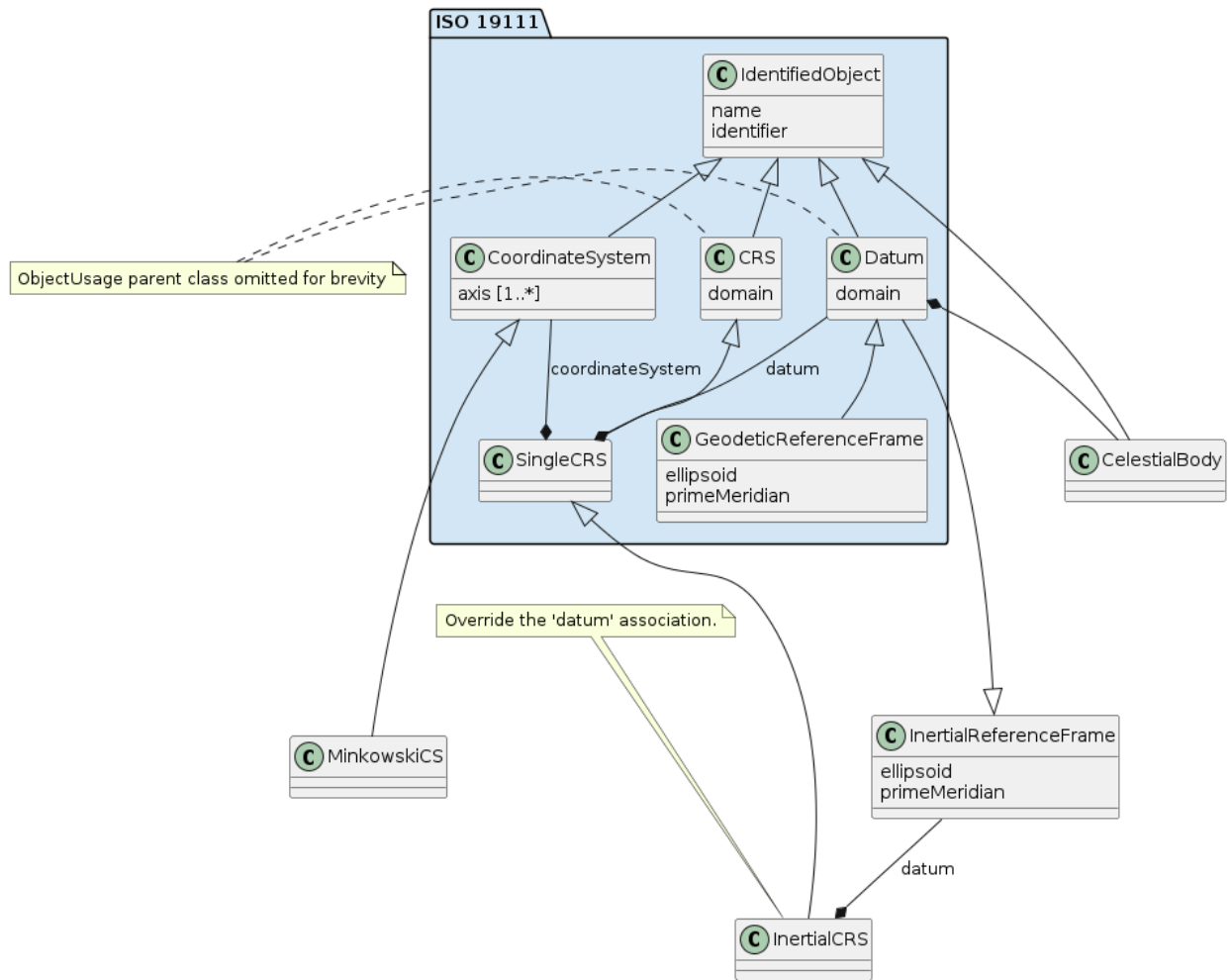


Figure 1

MinkowskiCS is introduced as a spatiotemporal equivalent of CartesianCS. Note that this is defined as a coordinate system (CS), not a coordinate *reference* system (CRS). A coordinate system is a purely mathematical object. It does not specify the location of the origin, or the precise orientation of axes relative to stars (a CS contains approximate axis directions, but only as help for axis identifications). Coordinate System do not contain empirical data such as geoid surfaces or gravity fields. Those information are rather represented by the *reference frame* or *datum* objects.

2.1.1. Discussion on keeping the Coordinate Systems simple

An ISO 19111 coordinate system is only a set of axes at no particular location, together with geometry formulas that are implied by the coordinate system type. For example, the use of a CartesianCS implies that angles, distances, surfaces, volumes, etc. are computed by formulas derived from Euclidean geometry. The use of a CylindricalCS or SphericalCS implies different sets of formulas for computing geometric properties. But all CoordinateSystem types defined by ISO 19111 imply relatively simple and well-known formulas. This simplicity is necessary for making possible to perform tasks such as solving the equations for computing intersections,

or solving integral equations for computing surfaces. The most complex coordinate system type defined by ISO 19111:2019 is the `EllipsoidalCS`. In that latter case, solving the integral equations requires series expansions, as done in map projections or geodesic calculations. Extensions to ISO 19111 such as `MinkowskiCS` should not be much more complex.

The simplicity mandated by above paragraph may seem restrictive, but actually it is an unavoidable step applied even with complex topographies. Complexity is usually managed by splitting a surface into small cells. Inside each cell, the “simple” mathematics of a coordinate system is assumed a good approximation. For example, the figure below zooms on a single cell of a complex surface. If a software computes the shortest path (the geodesic) between two points inside that cell by applying linear interpolations between (x_1, y_1, z_1) and (x_2, y_2, z_2) coordinate tuples, then that software is implicitly using a Cartesian coordinate system. The cells size (grid resolution) is chosen in such a way that approximating the real surface by a flat surface inside the cell area is considered an acceptable error.

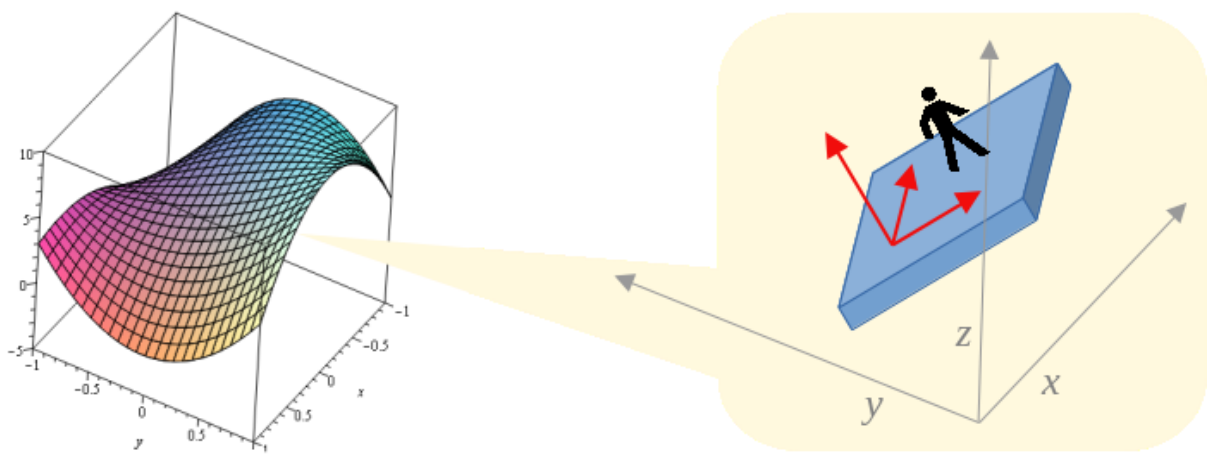


Figure 2 — Surface with local Cartesian CS in each cell

In above example, the shortest path inside a cell can be computed using the global Cartesian coordinate system for the whole grid (gray axes) or a Cartesian coordinate system which is local to the cell (red axes). Both coordinate systems produce the same results in this particular example, but the local CS is simpler because it can be reduced to a two-dimensional coordinate system (i.e. the perpendicular axis can be omitted) if all points lie on the surface. Coordinate transformations between those two systems are not discussed in this section, as they require Coordinate Reference Systems (CRS) instead of Coordinate Systems (CS).

The local coordinate system does not need to be of the same type than the global one. For example, the local CS could be an `EllipsoidalCS` if the latter is a better approximation of the real geometry inside the area of that cell. This is illustrated in the figure below, and in this case doing geodesic calculations in the local CS produces different results than the cases illustrated in Figure 2.

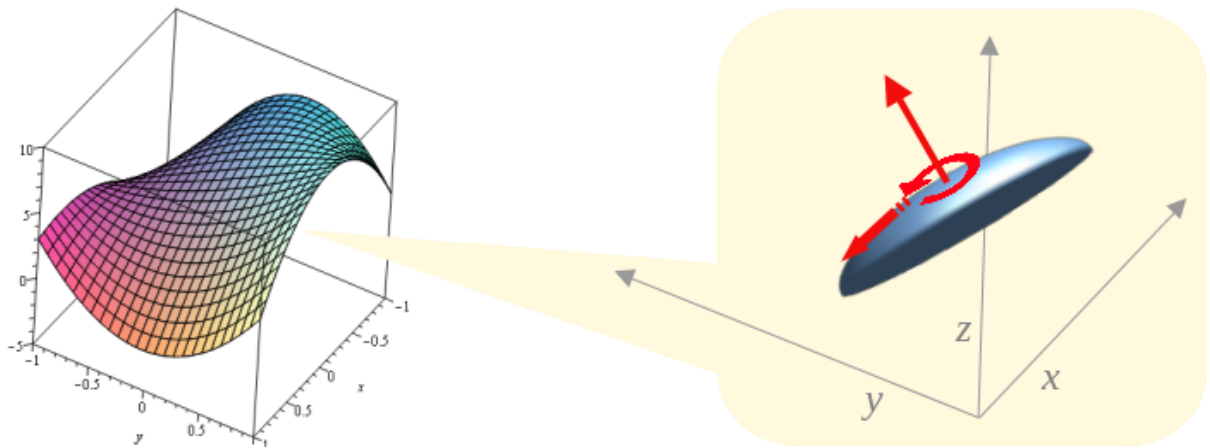


Figure 3 — Surface with local ellipsoidal CS in each cell

In above examples, calculations must be limited to a single cell. If the feature of interest spans more than one cell, then calculations must be performed separately in each cell and the results combined in some way. ISO 19111 does not specify how multi-cells calculations should be done.

An application can choose to use the same `CoordinateSystem` (CS) for all cells, even if axis directions relative to stars vary slightly between different cells. Using a common CS means that all cells use the same geometry formulas with axes having the same units of measurement and the same orientations relative to the cell. The precise orientations of axes relative to stars is encapsulated in the Reference Frame, which is not part of a `Coordinate System` (it is part of a *Coordinate Reference System*). Using an uniform `Coordinate System` makes easier to combine the results of adjacent cells, for example, by stating that all cells use a `CartesianCS` with measurements in meters. So volumes or areas computed in adjacent cells can be summed without unit conversions. This is true even for `EllipsoidalCS` using ellipsoids of different sizes and shapes, because in the ISO 19111 model, `Ellipsoid` is also a *Reference Frame* property rather than a CS property.

2.1.2. Discussion on CRS domain of validity versus CS simplicity

An ISO 19111 *Coordinate Reference System* (CRS) is associated to exactly one `Coordinate System` (CS), omitting Compound CRS for this discussion. A CRS is also associated (indirectly, through datum identification) to information about the CRS origin and “absolute” axis orientations (e.g. relative to stars). Consequently each cell in a grid such as Figure 2 or Figure 3 has its own local CRS, because each cell has a different reference frame for different origin and “absolute” axis directions.

Coordinates can be transformed from the global CRS to the local CRS of a particular cell before to compute geometric properties in the area of that cell. Note that in this scenario, the complexity of the irregular surface appears in the process of *transforming* coordinates from the global CRS to a local CRS. This complexity does not appear in any coordinate system before or after the transformation. In ISO 19111 modeling, the complexity is in `CoordinateOperation`, not in `CoordinateSystem`.

Given the restriction that each ISO 19111 CRS (excluding CompoundCRS) shall be associated to exactly one CS, and given that a CS should be mathematically simple, the CRS in Figure 2 and Figure 3 can be based on one of the following choices:

1. The global three-dimensional Cartesian coordinate system, in which case the complex surface play no role. That surface is a feature *inside* the CS domain, but does not define the CS itself and is not involved in any calculation between pairs of (x, y, z) coordinate tuples.
2. A global two-dimensional coordinate system resulting from a curve-fitting of the complex surface. The result can be the surface of a plane, cylinder, sphere, ellipsoid, or something else defined outside ISO 19111. That approximation can be very good in some cases (e.g. Earth surface approximated by an ellipsoid). This option is nevertheless not discussed further in this Engineering Report.
3. A local two dimensional coordinate system on the complex surface, i.e. the CS of one of the cells. That coordinate system may be Cartesian or ellipsoidal (among others) as illustrated in the two above examples, depending on the desired compromise between accuracy and mathematical simplicity.

Each alternative has pros and cons. Option #1 allows the referencing of any point anywhere in the domain covered by the global CRS. However, it does not allow accurate geodesic calculations (e.g. shortest path on the surface) between pairs of points. Conversely, option #3 is limited to a small area, but allows much more accurate geodesic calculations in that area.

Note also that complex numerical models, e.g. based on fluid mechanic (meteorological and oceanographical models) or based on the equations of Einstein's general Relativity, do not change the fundamental approach. Those numerical models still need to split a large space into a grid of cells small enough for allowing the use of "simple" mathematics inside each cell with acceptable errors. The grids may be more sophisticated than grids of equilateral cells, but they are still grids with cells delimited by analytic functions.

Above discussion implies that if geometric properties or geodesics paths are wanted with high accuracy, then their calculations may require a CRS restricted to a local area. However, the area can be made larger by a choice of coordinate system type, or by a choice of kind of acceptable errors. This is partially illustrated by the figure below with one-dimensional coordinate systems, but the discussion should be thought as if the coordinate systems were two- or three-dimensional. The left side shows the case when Cartesian coordinate systems is wanted. No CartesianCS can be a good fit for the whole mountain, so they have to be restricted to smaller areas. The calculations can only be done between pairs of points on the same side of the mountain. If a calculation needs to be done between two points that are on each side of the mountain, it needs to be split in two parts computed independently on each side, then combined somehow. In the ISO 19111 model, those two computations are done in two different Coordinate Reference Systems. However, if the coordinate system is based on a quadratic curve instead of straight lines, as shown in the right side of the figure below, then it becomes possible to do computation on the whole mountain with a single CRS. Computation between points on different sides of the mountain is no longer a problem.

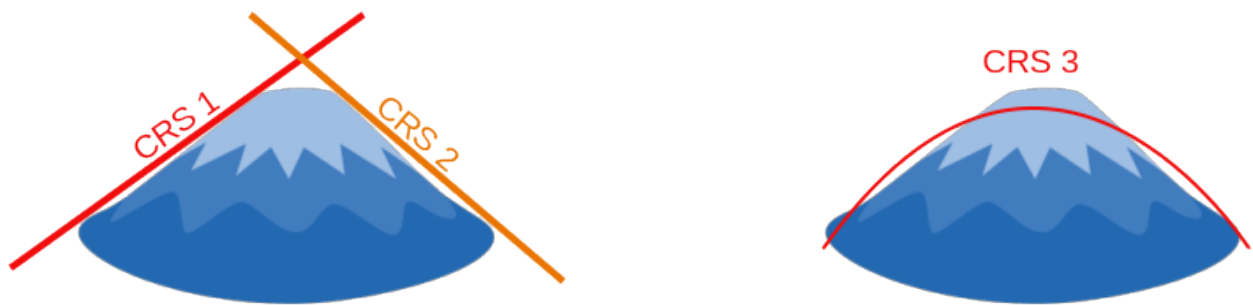


Figure 4 – More complex coordinate system covering larger area

In a two-dimensional coordinate system, the use of `EllipsoidalCS` instead of `CartesianCS` allows to cover a larger surface in a way similar to Figure 4. In the case of Earth, using `EllipsoidalCS` is often an approximation good enough for covering the whole planet with a single CRS, e.g. WGS84. However, it does not change the general principle that `EllipsoidalCS` is still an approximation. On some irregular bodies, that approximation may not be good enough for allowing the use of a single `EllipsoidalCS` for the whole body with the desired accuracy.

2.1.3. Coordinate System versus Coordinate Transformation roles in accuracy

A coordinate system based on a cubic curve would be more accurate or would cover a yet larger area than the examples shown in Figure 4, but at the cost of more mathematical complexity. That mathematical complexity cannot be increased indefinitely, because the equations for intersections, surfaces, etc. can quickly become unsolvable. For example, ellipsoidal equations are only of degree 2, and already very difficult to solve. In order to cover a larger area, an alternative to increasing mathematical complexity is to keep a simpler coordinate system, such as `CartesianCS`, but accept some errors. For example, it is possible to favor accuracy in surface calculations at the expense of accuracy in angles, or conversely. Controlling those errors is the art of map projections.

The overall accuracy does not depend only on the accuracy of formulas in the chosen coordinate system. It depends also on the accuracy of the transformation used to get there. For example, Figure 5 below illustrates the results of a numerical experiment done at a point located at 45°N on Earth. The following test was repeated at distances varying from 1 meter to 1000 kilometers around the above-cited point:

- A circle is drawn around the center point (0°E 45°N) with a radius equals to the distance to test. The circle is determined by computing the locations of 360 points (each point is separated by 1°) using geodesic formulas on ellipsoid published by Karney (2013).
- Distances and azimuths are computed on paths from the center point to the above-cited 360 points using the following methods:
 - Using `SphericalCS` formulas (distances are the arc lengths):
 - On an authalic sphere (sphere of the same surface than the ellipsoid) using the geographic coordinates as-is.

- On the same authalic sphere, but with geographic coordinates converted from the geodetic latitudes on the flattened sphere (Earth) to the geocentric latitudes on the authalic sphere. Those latitudes differ by about 21 kilometers. This operation is identified as a “datum shift” in Figure 5.
- Using CartesianCS formulas (distances computed by the Pythagorean formula):
 - After projection to “Mercator (variant B)” with standard parallel at 45°.
 - After projection to “Lambert Conic Conformal (2SP)” with the two standard parallels at 45°.
- Above calculations are compared with the expected distances and azimuths, which are the values used for drawing the circle in the first step.

The Y axis below shows the relative error, as a percentage of the distance shown on the X axis. For example, an error of 0.25% for a distance of 10 kilometers is an error of 25 meters. The error bars correspond to one standard deviation.

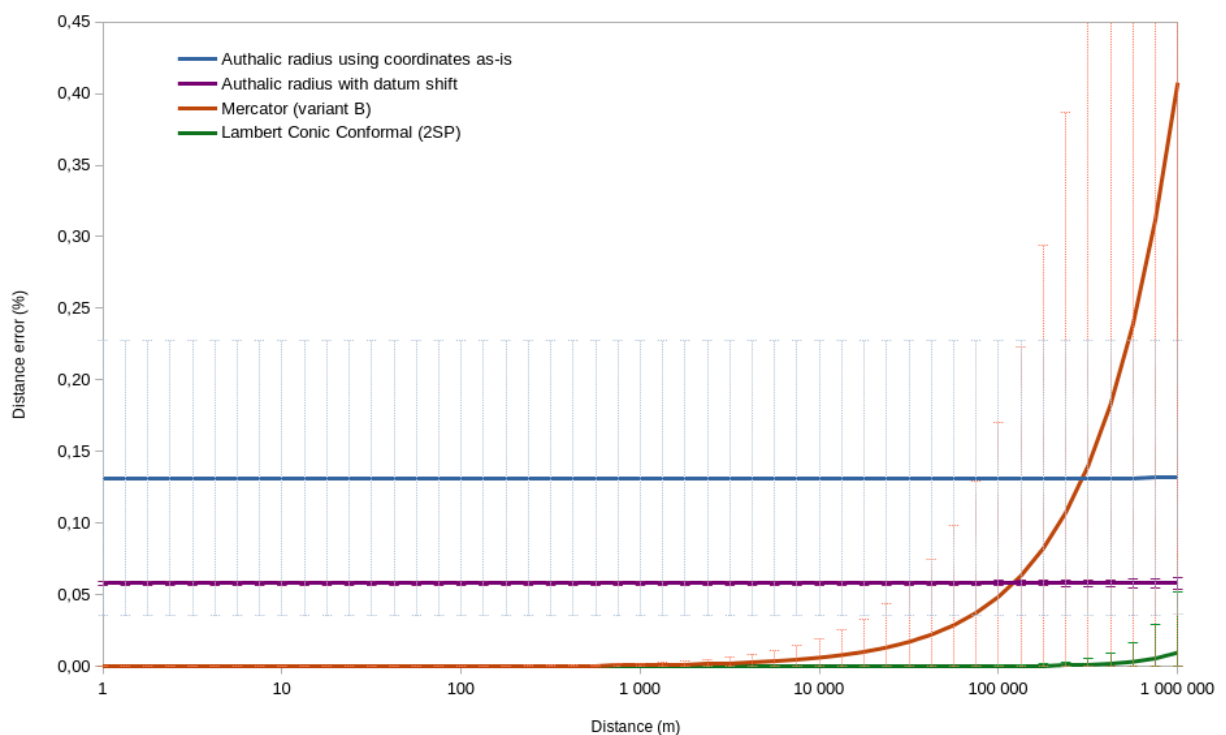


Figure 5 — Relative errors of distances around 0°E 45°N computed with different methods

We observe that naive application of spherical formulas on Earth, with WGS84 latitudes and longitudes used directly, is not very good compared to other approaches. The relative errors is constant at around 0.13% over the range of tested distances, and the standard deviation is almost as large. A more sophisticated calculation converting the latitude values before to apply spherical formulas gives better results, with relative errors at around 0.06% and negligible

standard deviations. However, maybe counter-intuitively, the use of CartesianCS gives better results than SphericalCS over small distances. Two factors contribute to that accuracy:

- The use of “standard parallel” projection parameter configures the map projection for optimal accuracy around the latitude used in this test, namely 45°N.
- The Mercator and Lambert Conic Conformal map projection formulas take in account the flattened shape of Earth.

On distances small enough, the geometrical distortions caused by the use of CartesianCS are smaller than the errors caused by ignoring Earth flatness in spherical formulas, or caused by the use of a global sphere radius which may not be optimal for the test area. (Note: a sphere with geocentric radius computed at 45°N was also tested, but the results are not shown because they were similar to the authalic sphere case). In this test, the use of “Mercator (variant B)” projection is more accurate than spherical formulas for distances up to 100 kilometers, and the use of “Lambert Conic Conformal (2SP)” is still more accurate even at 1000 kilometers.

The results of using “Popular Visualisation Pseudo Mercator” are not shown in this graph, because the errors are around 41% and would not fit in this Y axis scale. Those errors are caused mostly by the scale factor not well suited to 45°N. Other factors contributing to the errors are the facts that this projection ignores Earth flatness, and uses the equatorial radius instead of some globally better value such as the authalic radius.

2.1.4. Implications for MinkowskiCS

MinkowskiCS is not applicable in the context of general relativity, but can be an approximation good enough at short distances from the object of interest. This is similar to CartesianCS in map projections, which are flat approximations of a curved surface considered good enough inside a known region. As shown in Figure 5, using such approximations can sometimes be more accurate than attempts to use coordinate systems that are better fits of the local geometry. This is because the overall accuracy depends on both the coordinate system fitness and the accuracy of the coordinate operation applied to get there.

In the context of Einstein’s relativity, the space-time geometry is described by a metric. The metric can be represented as a matrix and defines how distances are computed. While those metrics are of particular importance in relativity, this concept can be used in classical geometry as well. The Euclidean metric used in CartesianCS is the simplest:

$$g_{ij} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (1)$$

Lets p be the coordinate tuple of a point in space or space-time, and p_i (or p_j) be one of the coordinates of that tuple. For a small displacement dp , the small distance ds is given by:

$$ds^2 = \sum_i \sum_j g_{ij} \cdot dp_i \cdot dp_j \quad (2)$$

For the Euclidean metric, the development gives the familiar Pythagorean formula:

$$ds^2 = \begin{pmatrix} +dx \cdot dx + 0 + 0 \\ +0 + dy \cdot dy + 0 \\ +0 + 0 + dz \cdot dz \end{pmatrix} = dx^2 + dy^2 + dz^2 \quad (3)$$

By comparison, the metric at the surface of a SphericalCS is as below, assuming a radius of 1, coordinates in (latitude, longitude) order and units in radians. The main thing to note is that this metric is not constant: the matrix varies at every position, because it depends on the latitude ϕ . Consequently computing the geodesic distance s^2 on a non-short path requires resolving an integral equation (not done in this ER).

$$g_{ij} = \begin{bmatrix} 1 & 0 \\ 0 & \cos(\phi) \end{bmatrix} \quad (4)$$

MinkowskiCS can be seen as CartesianCS generalized to the spatiotemporal domain. Its metric is shown below, using the (- + + +) sign convention. Coordinates are in (t, x, y, z) order with the time coordinate already multiplied by c, as required by the MinkowskiCS definition proposed in this ER:

$$g_{ij} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

MinkowskiCS may not provide sufficient accuracy in some situations, in the same way as CartesianCS on Earth's surface is not accurate anymore when the area of interest become too large. The accuracy on Earth can be improved by “upgrading” CartesianCS to EllipsoidalCS. Likewise the accuracy in a relativist context may be improved by “upgrading” MinkowskiCS to another relativist coordinate system using a more complex metric. However, those “upgrades” cannot be such as the geometry formulas become unsolvable. When the combination of large region and high accuracy exceeds what can be solved, splitting the region in smaller cells as illustrated in Figure 2 become unavoidable.

This Engineering Report does not explore how to combine calculations done in adjacent cells. Doing so may require the participation of other OGC working groups such as Discrete Global Grid Systems (DGGS). However, those combinations would probably require coordinate transformations between cells. This is where the complexity of geoid surfaces or gravitational fields are involved. But this complexity is in the *transformation* between two CRS, not in the CRS themselves.

2.2. Additions to ISO 19111

Most classes listed below are new and do not require modifications to the ISO 19111 Standard, as they can be specified as an extension in a separated document. An exception is `CelestialBody` which may need to be included in ISO 19111 if an association from `Datum` to `CelestialBody` is added.

2.2.1. InertialCRS

Description: a 2-, 3- or 4-dimensional coordinate reference system with axes at fixed position relative to stars.

Properties: this class inherits the attributes and associations defined by `SingleCRS` with no addition. However, the datum association is restricted to the `InertialReferenceFrame` subtype. Such restrictions are called *type covariances*. The association is repeated below for specifying this restriction.

Table 3 — InertialCRS properties

PROPERTY	TYPE	OBLIGATION	DESCRIPTION
datum	InertialReferenceFrame	Mandatory	Association to the inertial datum used by this inertial CRS.

2.2.2. InertialReferenceFrame

Description: an inertial datum defining the precise location and orientation in 3- or 4-dimensional space of a defined ellipsoid (or sphere), or of a Cartesian (3D case), or of a Minkowski (4D case) coordinate system centered in the ellipsoid (or sphere).

Properties: inertial reference frames have the same properties than geodetic reference frames, except that the ellipsoid and the prime meridian are optional. The prime meridian does not need to be specified because it rotates with the celestial body, and therefore cannot be used as a reference in an inertial frame.

Table 4 — InertialReferenceFrame properties

PROPERTY	TYPE	OBLIGATION	DESCRIPTION
celestialBody	CelestialBody	Optional	Association to the celestial body used by this inertial datum.
ellipsoid	Ellipsoid	Optional	Association to the ellipsoid used by this inertial datum.
primeMeridian	PrimeMeridian	Optional	Association to the prime meridian used by this inertial datum.

2.2.3. CelestialBody

Description: a name or identifier of the star, planet, asteroid or other celestial body for which a reference frame is defined.

Properties: since `CelestialBody` is an `IdentifiedObject`, it inherits the name and identifiers properties. The celestial body shall be identified by its name, optionally with aliases, and optionally one or many identifiers valid in some registries.

2.2.4. MinkowskiCS

Description: a 4-dimensional coordinate system. The three spacial dimensions give the position of points relative to orthogonal straight axes. The fourth dimension gives the position in time since an epoch. All axes shall have the same length unit of measure, with time expressed as the distance covered by light in the vacuum during the elapsed time. A `MinkowskiCS` shall have exactly four axis property elements.

Properties: no additional properties compared to all other `CoordinateSystem` classes.

2.3. Changes to ISO 19111

This section describes proposed changes to ISO 19111 abstract model. If accepted, those changes would require the publication of an ISO 19111 corrigendum.

2.3.1. Association to CelestialBody

The association to `CelestialBody` should be defined in the base `Datum` class for allowing the celestial body to be defined for any kind of datum: not only inertial, but also geodetic, vertical, parametric, and more. Doing so would require the addition defined in Clause 2.2.3 to be included in ISO 19111.

The association is defined as optional for compatibility with existing CRS definitions. If the celestial body is not provided, the default value is unspecified. It may be Earth, or it may be inferred from the context.

2.3.2. DomainUsage

In the ISO 19111 model, most classes inherit a set of properties from the `IdentifiedObject` parent class. Those properties are *name*, *aliases*, *identifiers* and *remarks*. But there is also one additional property, namely *domains*, which is inherited only by the `Datum`, `CRS` and `CoordinateOperation` classes. ISO 19111:2019 does this restriction by defining those properties in an intermediate class named `ObjectUsage`. ISO 19111:2007 did this restriction by repeating properties in the three above-cited classes, in a way similar to the UML of section Clause 2.1.

The following UML shown the ISO 19111:2019 model. The `ObjectUsage` contains only a domain property and is inserted between the `IdentifiedObject` parent class and the `Datum`, `CRS` and `CoordinateOperation` subclasses. It was done that way for preserving the same

restriction as in ISO 19111:2007, where only Datum, CRS and CoordinateOperation could have a scope and domain of validity.

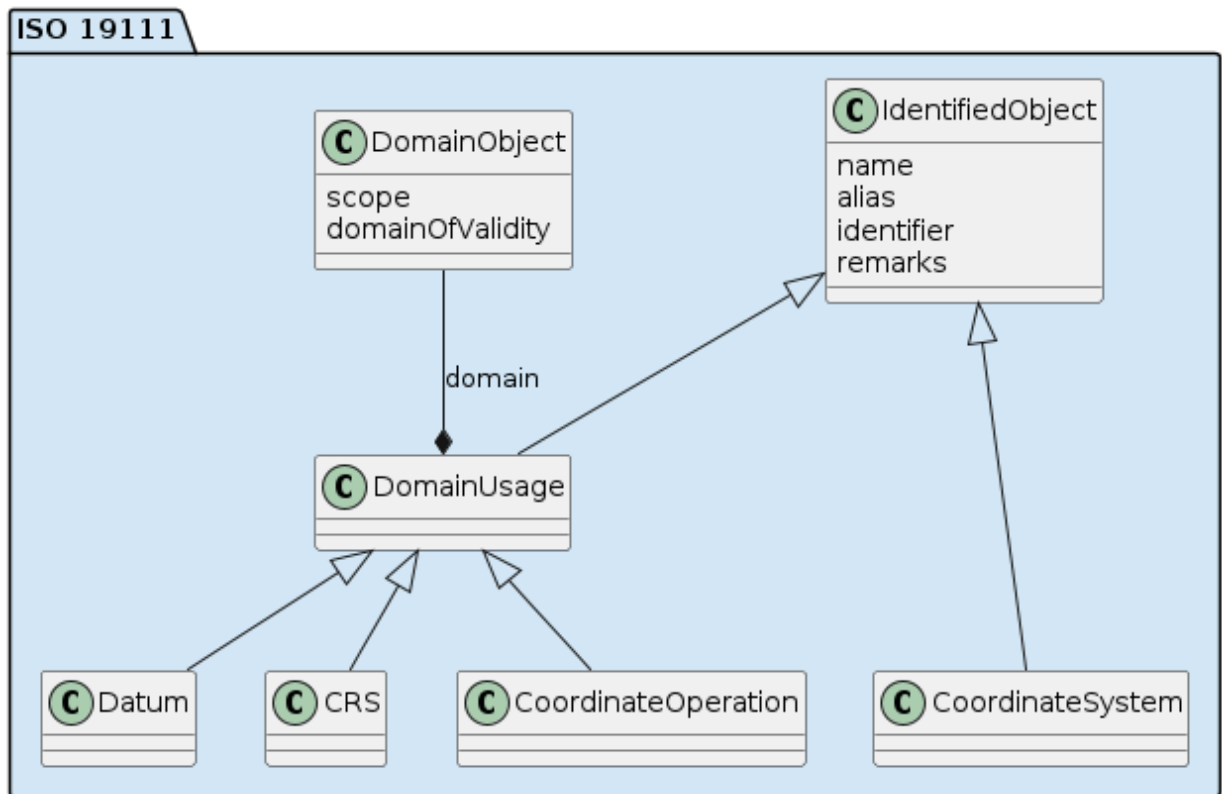


Figure 6

We propose to remove the restriction saying that only Datum, CRS and CoordinateOperation can have a scope or a domain of validity. We think there is nothing inherently wrong in allowing to specify a scope on most kinds of CRS-related objects. For example:

- A MinkowskiCS definition may want to specify “*for use with special relativity*” as the scope.
- An Ellipsoid definition may want to specify a limited geographic domain of validity if the celestial body has an irregular shape with different ellipsoidal approximations for different geographic areas.
- A PrimeMeridian definition may want to specify a limited temporal extent if it determined by an ephemeral feature, for example of the surface of a gaz planet.
- A CoordinateSystemAxis with direction such as “South along 60°E meridian” way want to specify “*for use at North pole*” as the scope.

The restriction can be removed by deleting the `ObjectUsage` class (not to be confused with the `ObjectDomain` class, which is kept unchanged), and moving the `domain` attribute up to the `IdentifiedObject` parent class.

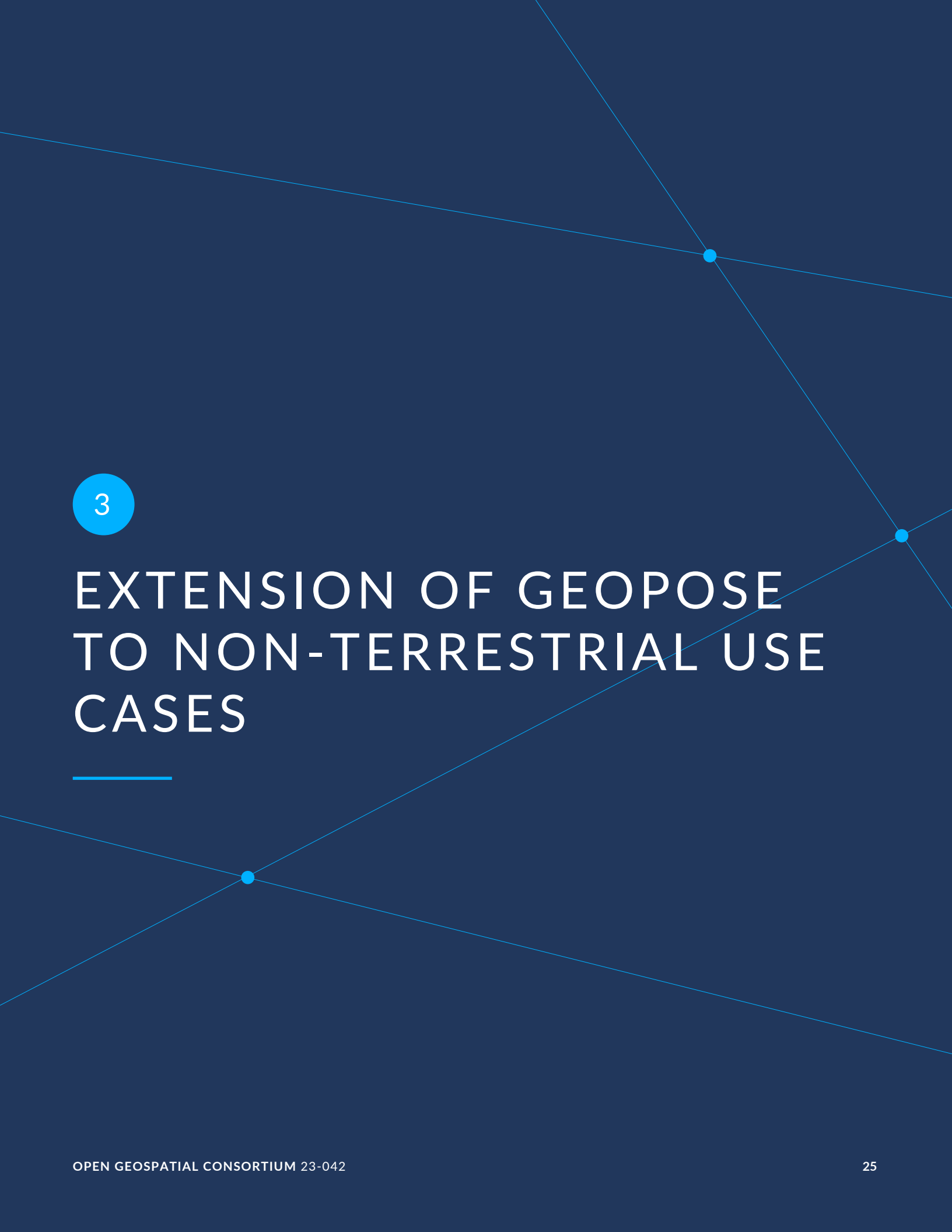
2.4. Example using DART data

The Standard extensions proposed in this Testbed are tested with data from the DART mission. The Double Asteroid Redirection Test (DART) was a mission from the [NASA Planetary Defense Coordination Office](#). The mission measured the changes of Dimorphos orbit around Didymos after an intentional collision of a spacecraft on Dimorphos surface. This Testbed encapsulated the some DART data into “extended ISO 19111” objects. The results are in the following files available on [GitLab](#). **TODO: replace by a public repository.**

- `Didymos.xml`: Inertial CRS centered on the Didymos main body.
- `Dimorphos.xml`: Insertial CRS centered on the satellite.

Snippets of above files are provided in Annex B.1.

TODO: application example



3

EXTENSION OF GEOPOSE TO NON-TERRESTRIAL USE CASES

EXTENSION OF GEOPOSE TO NON-TERRESTRIAL USE CASES

While the GeoPose 1.0 standard [ref] is described in terms of traditional Earth-based coordinate systems, adaptation to non-terrestrial use cases is straightforward, even for distances, relative speeds, and accelerations where the absolute Galilean space and time of Newtonian mechanics does not apply. The key is that the two components of a pose can be generalized to support a positioned entity that is oriented to another entity within its frame of reference.

3.1. GeoPose Conceptual Framework

It is an interesting accident that the computer graphics meaning of the term “pose” refers to the position of an observer and its orientation to another entity, while the non-technical concept of “pose” refers to the orientation of an observed entity to an observer:

A particular position in which a person stands, sits, etc. in order to be photographed, painted....

Figure 7

vs

The position and orientation of an object, usually in three dimensions

Figure 8

The two concepts are closely related and both are important in spatial computing. The first has to do with the location and orientation of a subject or observation. A natural name would be “subject pose”. The second has to do with the pose of an observer or imaging sensor. Natural names would be “observer pose” or “sensor pose”. The terms “subject”, “subject pose”, “sensor”, and “sensor pose” are used in the following discussion as follows:

“subject” “observer” “subject pose” “observer pose”

A Pose describes the position and orientation of an object (independently if it is real or synthetic in nature). A Pose is a special case of a (Frame)Transform system and, as such, facilitates the translation between two Reference Frames (without accounting for scale factors). A Fixed Pose is a Pose whose Outer Reference Frame is related to an Ephemeris Object — a spatial object whose position and orientation are externally defined. A GeoPose is a Fixed Pose related to a geospatial Ephemeris Object (by default, the Earth) via a Topocentric Reference Frame.

3.2. GeoPose Architecture V 1.0

3.3. GeoPose Suggestions from TB-18, Public Review, and Implementation

“NeoPose” is a temporary term applied to the experimental follow-on to GeoPose 1.0 with the goal of addressing five GeoPose issues in the context of the Hillyfields Bubble:

- Separate conceptual model, logical model(s), and encoding(s)
- Go beyond Json serialization and support transformations and other operations
- Subsume Proj and other libraries?
 - In general
 - For specific use cases
- Use a single logical model to support the GeoPose 1.0 use cases
- Define hierarchical or compositional relationship between Basic and Advanced targets
- Collapse composite targets into a single framework
 - Use plugin models — mediating generic objects — to
 - Support specific Temporal Coordinate Systems (TCS)
 - Support specific Spatial Coordinate Systems (SCS)
 - Support specific Temporal Coordinate Transformations (TCT)
 - Support specific Spatial Coordinate Transformations (SCT)
 - Support Event Coordinate System (ECS) as combined TCS and SCS
 - Support specific Event Transformations from Outer ECS to Inner ECS
 - Follow same pattern for rotations — note that Lorentz Transformation is “rotation-like”
- Support reliable and efficient plugin type discovery
 - Use “duck typing” to defer implement dynamic typing of parameters for transformations

- ECS parameters
- SCS parameters
- FrameTransform(event, Outer ECS, Inner ECS, transformation)
- RotationTransform(event, parameters, transformation)

Transformation can either validate parameters before calculation or use runtime error handling (e.g. exceptions) to handle errors during calculations. Need to define value for non-present objects in order to support the Basic GeoPoses with explicit specification of ECS objects and transformations. For example, in the Basic forms,

```
{
  "position": {
    "lat": 47.7,
    "lon": -122.3,
    "h": 11.5
  },
  "quaternion": {
    "x": 0.2303923729814588,
    "y": -0.03749706362805052,
    "z": 0.15620176329098345,
    "w": -0.9597470155390087
  }
}
```

Figure 9

The outer ECS has a null time value and expects to see a “location” object with WGS84 lat, lon, h values. The inner ECS has a null time value and 3 Cartesian LTP-ENU coordinates. The transformation is the WGS-84 to LTP-ENU transform. The rotation transformation has to make a dynamic determination of the type of the parameter object and proceed according to the YPR angle type or quaternion type. Dynamic typing makes it possible to add additional types (e.g. swing and twist or look-at) but the burden is on the transform to deal with them. Serialisation in Json for Basic GeoPoses looks the same as GeoPose 1.0. Advanced can be supported but does anyone care?

3.4. Implementing Selected Suggestions

3.4.1. Broadening the Formalism

3.4.2. Support External Standards for Coordinate Reference Systems

3.4.3. Support External Development of Reference Frame Transformations

3.4.4. Unified Galilean One- to Three-Dimensional Space and Time

Why dimensions 1-3 are practically important.

The question of time synchronization

3.4.5. Unified Galilean and Minkowski Spacetimes

- Special and general relativity.
- The question of time synchronization.
 - Einsten Clock Synchronization
- Geometric or Kinematic Effects of Relative Motion
 - Nonrelativistic
 - Classical Doppler Effect
 - Aberration
 - Light Travel Time
 - Relativistic

Especially when dealing with relativistic effects, there is a difference between how things are and how things appear. How they appear is the most relevant from a relativistic point of view. Both what is “real” and how they appear visually depend on independent definitions and physics.

One definition of what is real, from the point of view of one inertial reference frame when observing objects in another frame in uniform relative motion, are the transformed coordinates of physical objects in the observed frame at a single time in the frame of the observer. Another

is the projection of arriving photons scattered from physical objects in the observed frame. In the first case, calculation of the transformed distances would show a length contraction in the direction of relative motion. In the second, purely visual effects such as Terrell-Penrose rotation would reflect the paths of the arriving photons, giving the impression of a rotation. Both views of reality are valid and it is only necessary to precisely define the nature of the observation.

In particular, while length contraction can be calculated, it cannot be observed visually because it is exactly compensated for by the constant finite speed of light.

These considerations will appear again in both the task of aiming an imaging sensor from low earth orbit accurately at the Street Drone and in animated rendering of the environment and various objects moving during the data capture in the Hillyfields Bubble. The first depends on straightforward determination of the apparent rotation of coordinate axes via the Lorentz Transformation. The second is a visual effect that depends on the Lorentz-transformed coordinates of light scattering points in the observed frame at events at the corresponding light travel time in the past. The photons across the image arrive simultaneously in the observer frame but departed at different times from points in the observed frame.

- Length (Lorentz) Contraction+
- Time Dilation+
- Relativity of Simultaneity+
- Consequences
 - Relativistic Doppler Effect
 - Terrell-Penrose Rotation
 - Pointing along geodesics

3.4.6. Pointing Task

Determination of the departure and arrival directions of a geodesic between pairs of events in distinct inertial frames.

- Aim visual sensor on satellite in orbit at 750 000 km above Hillyfields $z = 0$ plane at LiDAR unit on Street Drone
 - About 140 seconds of visibility time
 - Approximate inertial reference frame for this time
 - Compute pointing angles with respect to spacecraft frame

3.4.7. Visualization Task

- Animate objects in Hillyfields Bubble

- In Galilean Space and Time
- In Minkowski Spacetime

$$\mathbf{z}' = \boldsymbol{\zeta} \tag{6}$$

$$\mathbf{x}' = \langle \boldsymbol{\eta} \times \boldsymbol{\zeta} \rangle \tag{7}$$

$$\int_a^b f(x) dx = \lim_{\Delta x \rightarrow 0} \sum_{i=1}^n f(x_i) \Delta x_i \tag{8}$$

In Minkowski spacetime

3.4.8. Application to Non-Terrestrial Use Cases

3.4.9. Experimental Results from the Hillyfields Bubble

- Proof of concept implementations for the NeoPose extensions
 - LEO Satellite Pointing Task
 - GeoPose Multiple Points-of-View in Minkowski Spacetime
- Repeatble results with full dataset
- NeoPose graphs

3.4.10. Implications and Recommendations for the OGC GeoPose Standard

3.5. Issues of locating terrestrial objects

TODO Chapter content.

3.6. An examination of experiments in recording the location of terrestrial objects

TODO Chapter content.

3.7. Road Network Use Cases

In Testbed-18, Away Team identified two road network use cases that could benefit by integrating WebVMT and OGC GeoPose with Moving Features from discussions with the UK National Highways Agency. Road traffic datasets can be enriched by aggregating geotagged video with sensor data observed from multiple sources concurrently, provided that these can be synchronized with sufficient accuracy.

3.7.1. Overview

In Testbed-19, Away Team proposed a plan to collect and aggregate geotagged video data suitable for analysis of the road network use cases using tools developed in previous OGC Testbeds:

- **Testbed-16:** Moving object previzualization was successfully demonstrated using WebVMT integrated with web technologies in a browser.
- **Testbed-17:** A suite of WebVMT tools was developed to track a cyclist from a moving vehicle using video and LiDAR data with GeoPose.
- **Testbed-18:** A 3D compass app was developed to demonstrate how GeoPose can be recorded using a smartphone, which was publicly released free of charge for Android devices.

3.7.1.1. Use Cases

Previous discussions with national road network authorities highlighted two use cases of particular interest:

- **Wrong way traffic** to rapidly identify and locate vehicles travelling in the opposite direction to the correct traffic flow which is a safety-critical issue. Being able to interview these elusive drivers can also provide valuable feedback to help reduce incident recurrence with targeted improvements to road junction design.
- **Litter monitoring** to assess accretion rates of objects that can hinder traffic flows by harvesting geotagged dashcam footage from a fleet of vehicles which routinely traverse

the road network for maintenance, delivery, or transportation purposes. Metrics can be used to plan and prioritize remedial action.

The essential elements of these use cases can be reduced to the following concept demonstrations.

- **Traffic cameras:** Associate observations of a moving vehicle seen by two static traffic cameras at different locations along a road.
- **Fleet survey:** Associate observations of a static object seen by dashcams from two moving vehicles passing the same location at different times.
- **Multi-view survey:** Associate observations of a static object seen from front- and rear-facing dashcams on a moving vehicle passing a location.

3.7.1.2. Data Capture

In April 2023, Open Site Plan and Away Team participated in a collaboration with Ordnance Survey at their Southampton headquarters to capture geotagged video from a selection of location-aware video devices. Footage was recorded from multiple devices concurrently — including smartphones, body-worn cameras, a drone and a StreetDrone Twizy autonomous vehicle — in order to study a number of use cases including the litter monitoring and wrong way traffic scenerios previously identified in Testbed-18. The aim was to show how accurately these data can be synchronized and demonstrate the value added by their aggregation.



Figure 10 — StreetDrone Vehicle Operated By Ordnance Survey

Geotagged video was recorded by two smartphones running Away Team's TrkdCam app which were mounted on tripods at the roadside. These acted as static traffic cameras to record details of passing vehicles for the wrong way traffic use case. The GeoPose of these devices was captured using the 3D Compass app from Testbed-18, together with reference photos to verify their locations.

Objects were placed at known positions by the roadside to act as targets for the litter monitoring use case, including common litter items such as metal drink cans and plastic bottles. In addition, the UK National Highways Agency supported this collaboration by lending motorway maintenance equipment which included temporary signs, metal A-frame stands for those signs and traffic cones with lights. The Agency has an operational requirement to quickly

identify and collect such items which are accidentally left behind after roadworks are finished as these are valuable assets that can be hazardous to road users when left in the wrong location.



Figure 11 — Roadsign A-Frame, Traffic Cone & Plastic Bottle

3.7.2. Analysis

Having captured the data, the initial challenge was to track passing vehicles from the roadside footage.

3.7.2.1. Vehicle Tracking

Moving objects were tracked from video using a process that was designed to take advantage of the oblique viewing angle by using lines of sight. This method is also robust to variations in the pan and tilt of the tripod mounting which inevitably occurred when the camera was removed to download files.

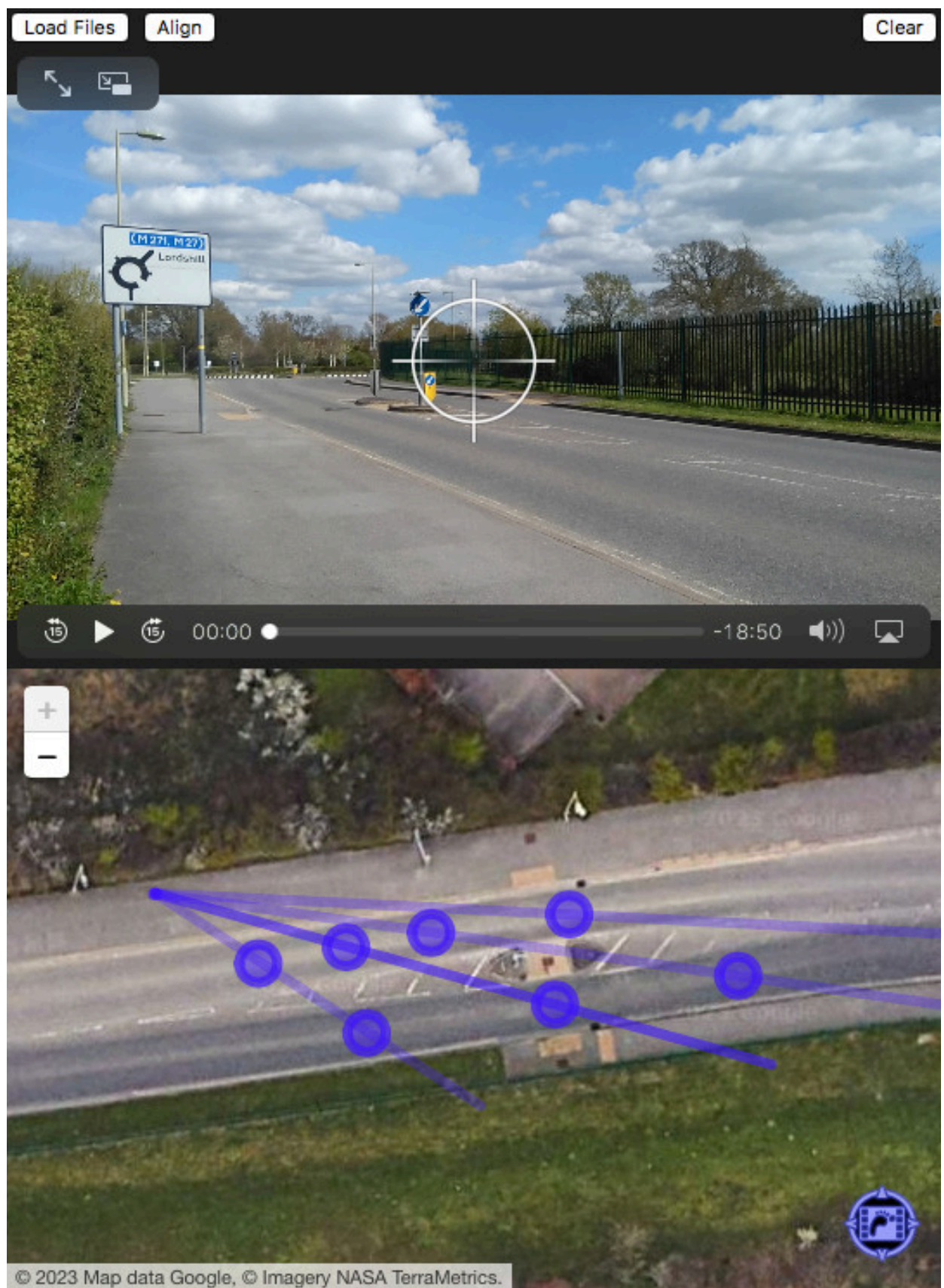


Figure 12 — Lines of Sight With Target Location Zones

The procedure can be summarised as follows:

1. Identify suitable lines of sight from the video frames.
2. Identify target locations corresponding to those lines.
3. Identify video frames with a moving object that matches those target locations.
4. Add a path segment to the corresponding WebVMT file.
5. Verify accuracy using previzualization in a web browser.

Figure 13 shows the observed track synchronized with video and a faint line indicating the camera's line of sight.



Figure 13 – Vehicle Tracking Previzualization

3.7.2.2. Synchronization Issues

The next challenge was to accurately synchronize data from different devices with a common timeline — in this case Coordinated Universal Time (UTC). Timing errors hinder the process of correctly associating observations of the same object at different times and locations between different devices. Hence, poor synchronization significantly reduces the value of data aggregation.

The smartphones used to record video from static tripods were linked to the cell network which enables access to accurate UTC time. However, smartphone cameras typically have a significant delay between a user request to start video recording and the first frame of that recording, and no accurate measurement of the video start time could be found. The latency depends on a number of factors including the camera hardware response time, resource pressure from other tasks and the device's power management strategy. As a result, the estimated start time was only accurate to within a few seconds.

By contrast, the StreetDrone vehicle was able to record timing data accurately to within 10ms and synchronized with UTC. A process was devised to match the trajectory recorded aboard the vehicle with the trajectory observed by the smartphone cameras in order to synchronize their video timelines with UTC. Visual analysis is limited to the unit time of a single video frame which is around 33ms at 30 frames per second.

Data were recorded natively in WebVMT by the smartphones and exported to WebVMT from the StreetDrone's inertial measurement unit (IMU). This enabled tools developed in Testbed-17 to be used for analysis and allowed results to be integrated with online maps for verification in a web browser.

A new WebVMT utility was added to the existing suite in order to calculate a list of closest approach times to a static location by a moving object. This tool can calculate results to millisecond accuracy and includes a radius parameter that enables multiple approaches to be discriminated correctly. By matching the StreetDrone trajectory data against the target locations used for vehicle tracking by the roadside cameras, an average time offset was calculated for each video file captured by the static cameras. In addition, the standard deviation was calculated to quantify the quality of the synchronization process. Results were used to adjust the WebVMT start time parameter in order to accurately synchronize the path segments observed by the static camera with UTC.

WebVMT files synchronized with UTC were clipped and merged using updated versions of the Testbed-17 tools suite to create previzualizations that demonstrate the accuracy of these results — see Synchronized Previzualization.



Figure 14 — Synchronized Data Previzualization

3.7.3. Results

3.7.3.1. Synchronization Metrics

The geotagged video files recorded had fairly consistent time offsets from UTC for each of the two smartphone devices — one was typically 1.5 seconds ahead and the other was around 2 seconds behind UTC. Quality metrics were good for both cameras since most standard deviations are in the range between 50ms and 200ms which is indicative of the average error.

Table 5 — Nokia 5 Synchronization Metrics

WEBVMT FILE	MEAN TIME OFFSET	STANDARD DEVIATION	SAMPLE SIZE
VID_20230425_145407.vmt	1.280	0.050	4
VID_20230425_151801.vmt	n/a	-	-
VID_20230425_171532.vmt	1.381	0.052	4
VID_20230426_120410.vmt	1.584	0.188	4
VID_20230426_130526.vmt	1.593	0.089	4
VID_20230426_150321.vmt	1.673	0.065	4

Table 6 — Moto G30 Synchronization Metrics

WEBVMT FILE	MEAN TIME OFFSET	STANDARD DEVIATION	SAMPLE SIZE
VID_20230425_145232.vmt	-1.755	-	1
VID_20230425_145348.vmt	-2.263	-	1
VID_20230425_150129.vmt	-1.915	0.154	2
VID_20230425_151132.vmt	-2.107	-	1
VID_20230425_151535.vmt	n/a	-	-
VID_20230425_171521.vmt	-2.278	0.138	4

WEBVMT FILE	MEAN TIME OFFSET	STANDARD DEVIATION	SAMPLE SIZE
VID_20230426_120319.vmt	-1.577	0.384	4
VID_20230426_130701.vmt	-1.959	0.052	4
VID_20230426_150316.vmt	-1.367	0.044	4

The best results represent a statistical error of less than 100ms (3 frames) with a 95% confidence level. In terms of road vehicles, this equates to a distance error of less than 1.3m at 50km/h (30mph) or 3.1m at 110km/h (70mph).

The accuracy of these results is confirmed by the previzualization screenshots shown in Synchronized Previzualization.

3.7.3.2. Wrong Way Traffic Use Case

The scheme devised to track passing vehicles from video footage could be easily extended to detect wrong way traffic.

Target location zones are sufficiently small to allow discrimination of individual lanes on a road. Using three or more zones per lane enables an automated detection routine to reliably determine in which direction a vehicle is travelling, and to discriminate between adjoining lanes which may have traffic flowing in opposite directions.

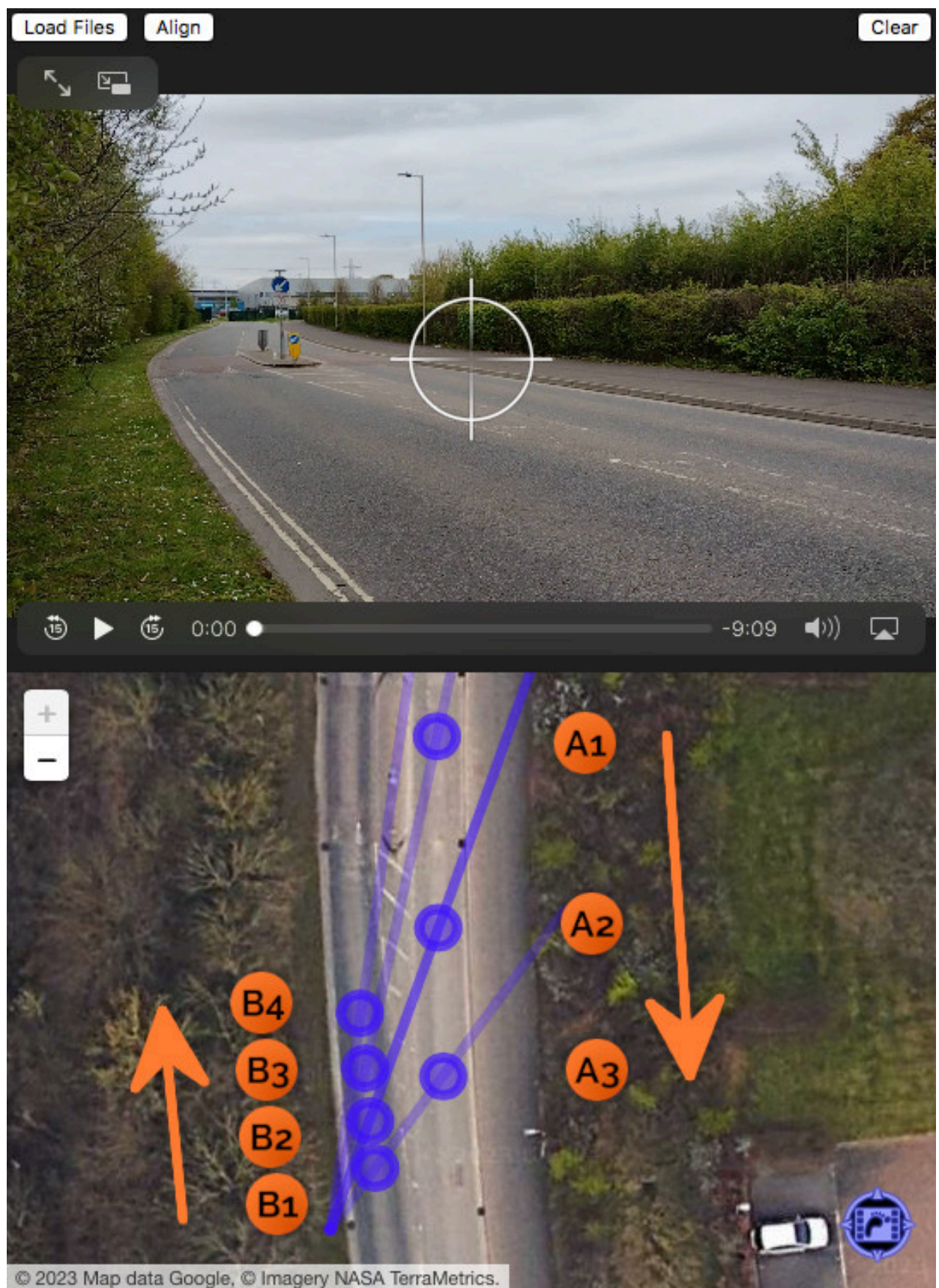


Figure 15 — Target Zones To Detect Wrong Way Traffic

Figure 15 shows the target location zones and lines of sight in blue. Detection labels for the zones in lanes A and B are shown in orange, and the arrows show the correct direction of traffic flow.

The expected sequence of detections for traffic travelling in the correct direction is either $A1 \rightarrow A2 \rightarrow A3$ or $B1 \rightarrow B2 \rightarrow B3 \rightarrow B4$ in this case – since vehicles drive on the left in the UK. Wrong way traffic would be detected by either $A3 \rightarrow A2 \rightarrow A1$ or $B4 \rightarrow B3 \rightarrow B2 \rightarrow B1$. This system could be easily adapted for use on a multi-lane road such as a motorway, but could also be used to police a no-overtaking zone on a two-way road.

An unexpected result was discovered during analysis which demonstrates the strength of this traffic monitoring scheme. The StreetDrone performed a U-turn which was captured by one of the static cameras. Although the algorithm was not designed to identify this manoeuvre, the incident was detected since the detection zones were triggered in an erroneous order that correctly indicates the event and demonstrates the value of this simple approach.

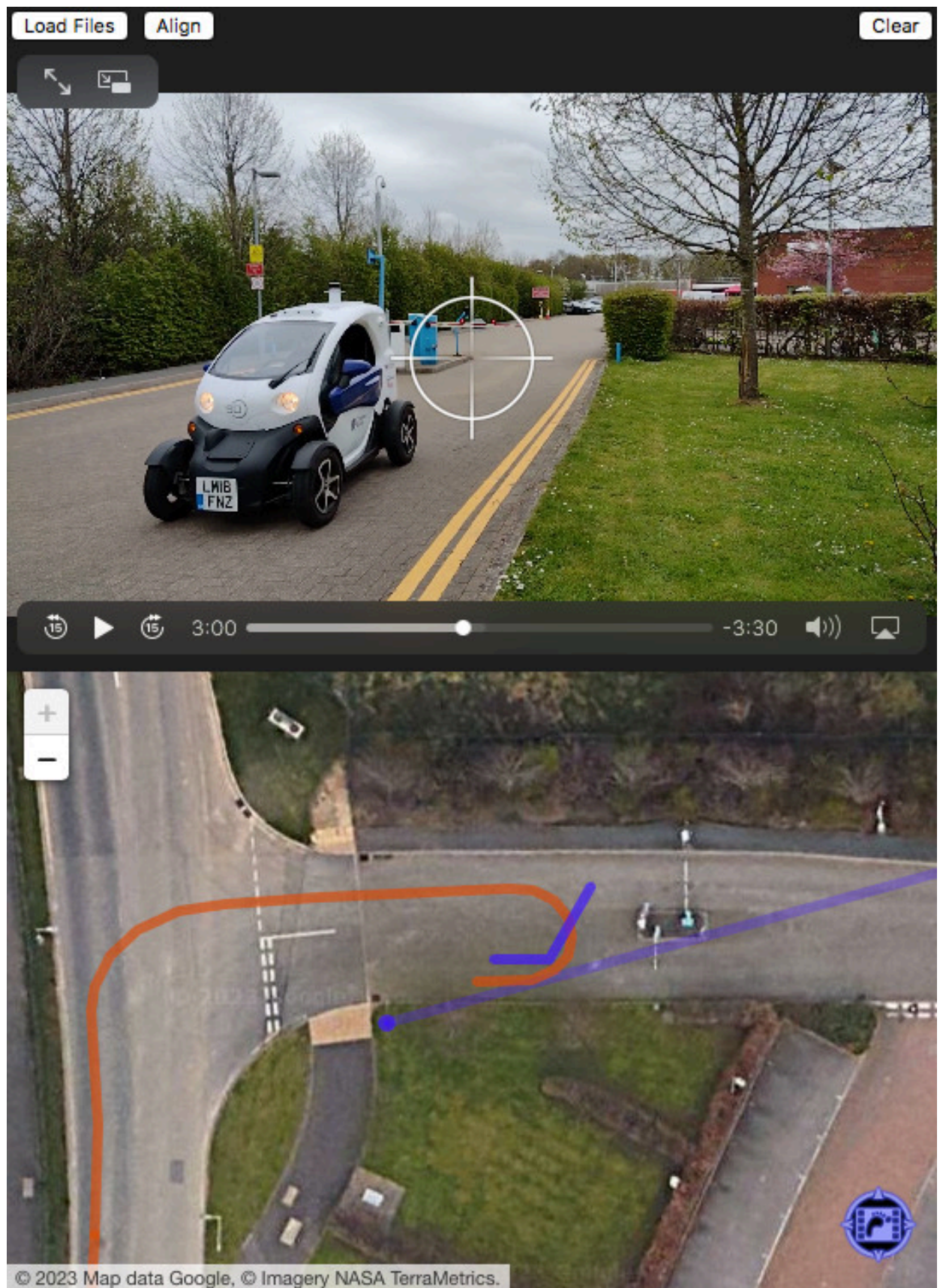


Figure 16 – Unexpected U-Turn Detection

3.7.4. Conclusions

Away Team has successfully demonstrated that moving objects can be used to synchronize data recorded by different devices with a high level of accuracy substantiated by quantifiable errors. This accurate synchronization enables aggregation of data from low-cost commercial video devices including dashcams, drones, body-worn cameras and smartphones that are geospatially aware.

A process has been identified to allow automated detection of wrong way vehicles using roadside traffic cameras which is an operational requirement for national road network agencies. In addition, passing fleet vehicles can be used to calibrate these video observations with UTC in order to aggregate them with and enrich other traffic monitoring data.

GeoPose and WebVMT analysis tools developed in previous Testbeds have been refined and enhanced to address these issues. This demonstrates the value of geospatial interoperability, and shows how geotagged video can be integrated with a web browser to leverage online technologies.

There was insufficient time in Testbed-19 to study the litter monitoring use case, though the data have been captured and this analysis could be deferred to Testbed-20 as described in Future Work.

3.8. Future Work

3.8.1. Litter Monitoring Use Case

Away Team captured data for the litter monitoring use case in April 2023, which could be studied in Testbed-20 since there was insufficient time to address this in Testbed-19.

Litter locations were identified and exported to WebVMT which could be matched against video and LiDAR data recorded by the StreetDone. Analysis could be performed using WebVMT tools developed during Testbed-17 for tracking static objects from a moving vehicle in order to monitor litter locations.

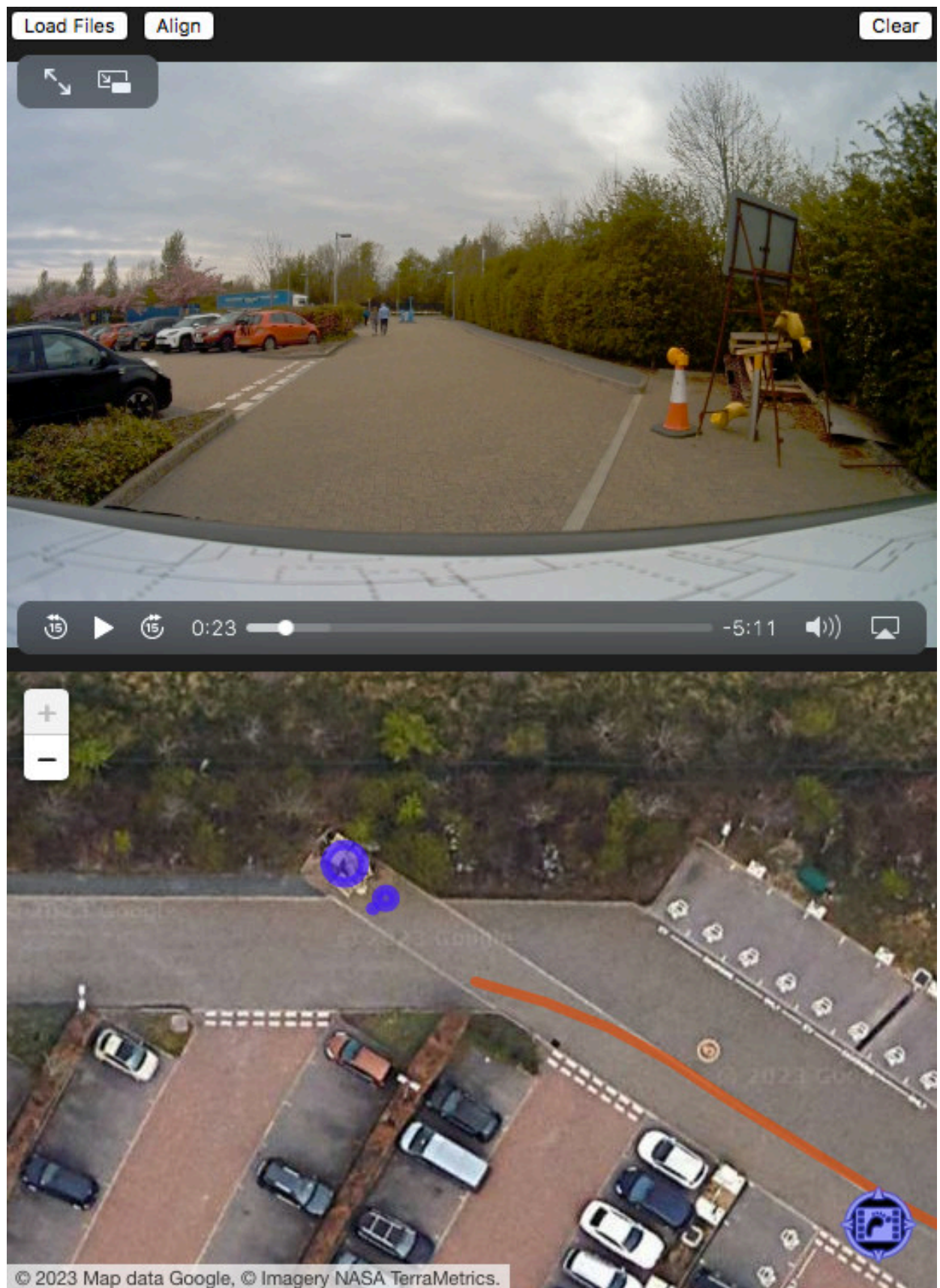


Figure 17 — Identified Locations For Litter Monitoring Use Case

In addition, video was concurrently captured by front- and rear-facing cameras on the StreetDrone to enable multi-view data aggregation. Hence the risk of other road vehicles obstructing the survey vehicle's view of a target location during a single pass can be mitigated in operational use.

Synchronizing data gathered by different devices and aggregating data from multiple passes of the same target location enables accretion rates to be calculated over time. This enables more accurate monitoring and prediction of litter levels so maintenance teams can be deployed more efficiently.

TODO Chapter content.



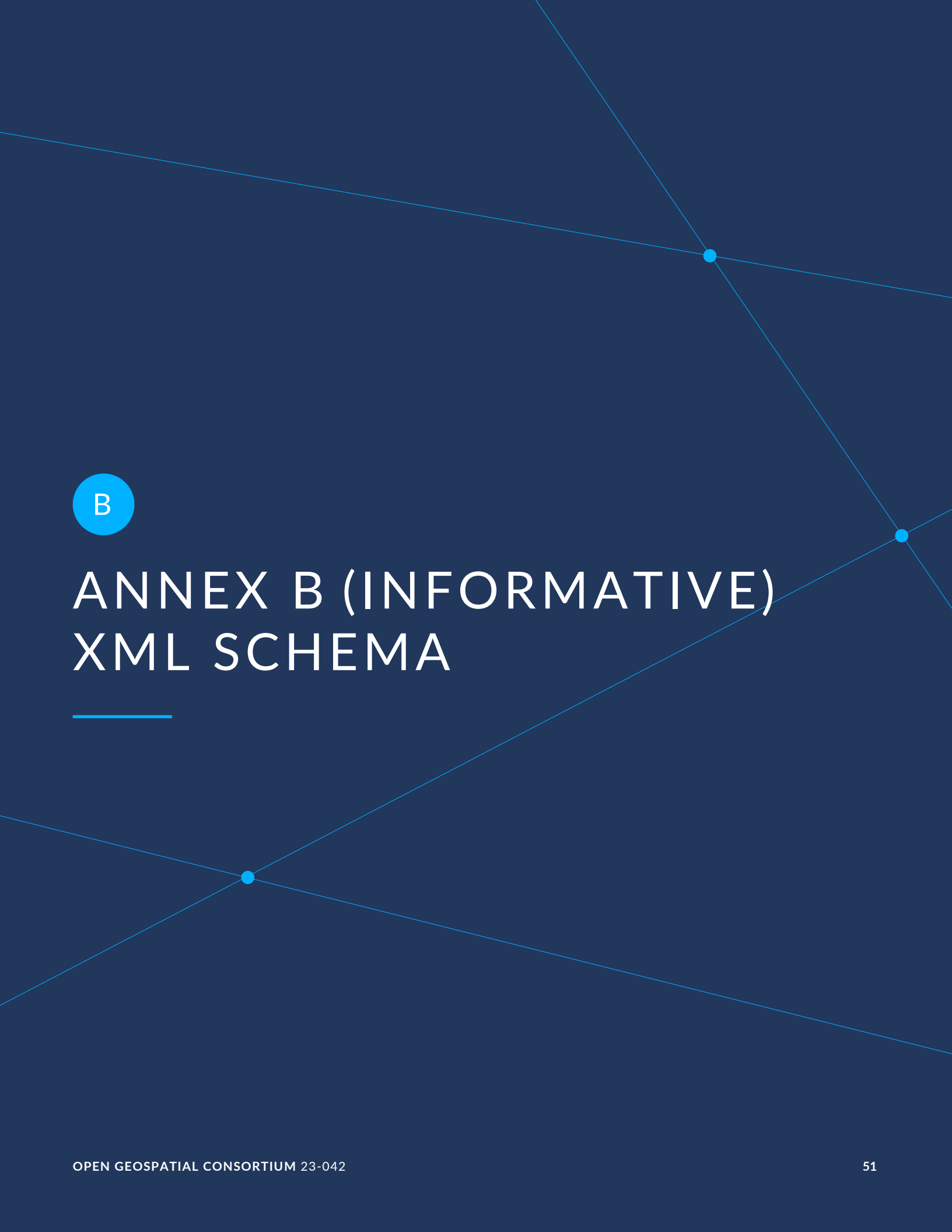
A

ANNEX A (INFORMATIVE) ABBREVIATIONS/ACRONYMS



ANNEX A (INFORMATIVE) ABBREVIATIONS/ACRONYMS

1D	One-dimensional
3D	Three-dimensional
4D	Four-dimensional
CRS	Coordinate Reference System
CS	Coordinate System
CSV	Comma Separated Values
ECEF	Earth-Centered – Earth-Fixed
ECI	Earth-Centered Inertial
GML	Geographic Markup Language
ICRF	International Celestial Reference Frame
ISO	International Organization for Standardization
OGC	Open Geospatial Consortium
UML	Unified Modeling Language
WKT	Well-Known Text



B

ANNEX B (INFORMATIVE) XML SCHEMA

B

ANNEX B (INFORMATIVE) XML SCHEMA

The following XML schema defines a GML extension with the new classes introduced in Clause 2.2. All GML extensions are done in the <http://www.opengis.net/gsp/1.0> namespace (non-standardized) and encoded in a single XML schema file named `geospatialInSpace.xsd`. The file is [available on GitLab](#). **TODO: replace by a public repository.** This schema can be used as a starting point for standardization.

This schema also contains one additional class, namely `TriaxialEllipsoid`, which is not listed in Clause 2.2. This is because ISO 19111:2019 already supports triaxial ellipsoids, but the standard GML schema is still based on the older ISO 19111:2007 model. Consequently `gsp:TriaxialEllipsoid` has to be added as an extension of `gml:Ellipsoid`, but only for the XML schema. This approach should not be replicated in abstract models because the way that `TriaxialEllipsoid` is defined below does not follow the Liskov Substitution Principle.

```
<?xml version="1.0"?>
<schema version          = "1.0.0"
        targetNamespace  = "link:++http://www.opengis.net/gsp/1.0"++[]
        xmlns:gsp        = "link:++http://www.opengis.net/gsp/1.0"++[]
        xmlns:gml         = "link:++http://www.opengis.net/gml/3.2"++[]
        xmlns             = "link:++http://www.w3.org/2001/XMLSchema"++[]
        xml:lang          = "en"
        elementFormDefault = "qualified">

  <annotation>
    <appinfo source="urn:ogc:specification:gml:schema-xsd:geospatialInSpace.
xsd:1.0.0">
      geospatialInSpace.xsd
    </appinfo>
    <documentation>
      The Geospatial in Space schema is an extension to the Coordinate
      Reference System
      components defined in the GML schema.

      This schema is not an OGC Standard. This schema is part of an OGC Public
      Engineering
      Report created as a deliverable in an OGC Interoperability Initiative
      and is not an
      official position of the OGC membership. It is distributed for review
      and comment.
      It is subject to change without notice and may not be referred to as an
      OGC Standard.
    </documentation>
  </annotation>
  <import namespace="link:++http://www.opengis.net/gml/3.2"++[]
        schemaLocation="link:++https://schemas.opengis.net/gml/3.2.1/
coordinateReferenceSystems.xsd"/>++[]
```

```

<!--
  Coordinate System
-->
<element name="MinkowskiCS" type="gsp:MinkowskiCSType" substitutionGroup=
"gml:AbstractCoordinateSystem">
  <annotation>
    <documentation>
      gsp:MinkowskiCS is a 4-dimensional coordinate system.
      The three spacial dimensions give the position of points relative to
      orthogonal straight axes.
      The fourth dimension gives the position in time since an epoch.
      All axes shall have the same length unit of measure,
      with time expressed as the distance covered by light in the vacuum
      during the elapsed time.
      A MinkowskiCS shall have exactly four gml:axis property elements.
    </documentation>
  </annotation>
</element>
<complexType name="MinkowskiCSType">
  <complexContent>
    <extension base="gml:AbstractCoordinateSystemType"/>
  </complexContent>
</complexType>
<complexType name="MinkowskiCSPropertyType">
  <annotation>
    <documentation>
      gsp:MinkowskiCSPropertyType is a property type for association roles
      to a Minkowski coordinate system,
      either referencing or containing the definition of that coordinate
      system.
    </documentation>
  </annotation>
  <sequence minOccurs="0">
    <element ref="gsp:MinkowskiCS"/>
  </sequence>
  <attributeGroup ref="gml:AssociationAttributeGroup"/>
</complexType>

<!--
  Triaxial ellipsoid
-->
<element name="TriaxialEllipsoid" type="gsp:TriaxialEllipsoidType"
substitutionGroup="gml:Ellipsoid">
  <annotation>
    <documentation>
      gsp:TriaxialEllipsoid is an ellipsoid described using an additional
      semi-median axis attribute.
      This attribute is for planetary applications and is not used when
      describing a bi-axial oblate
      reference ellipsoid model of the Earth. For a triaxial reference
      ellipsoid it is usual for the
      secondDefiningParameter to be the ellipsoid's semi-minor axis.
    </documentation>
  </annotation>
</element>
<complexType name="TriaxialEllipsoidType">
  <complexContent>
    <extension base="gml:EllipsoidType">
      <sequence>
        <element ref="gsp:semiMedianAxis"/>
      </sequence>
    </extension>
  </complexContent>
</complexType>

```



```

        </extension>
      </complexContent>
    </complexType>
    <element name="semiMedianAxis" type="gml:MeasureType">
      <annotation>
        <documentation>
          gsp:semiMedianAxis specifies the length of the semi-median axis of the
          ellipsoid, with its units.
          Uses the MeasureType with the restriction that the unit of measure
          referenced by uom must be suitable for a length,
          such as metres or feet.
        </documentation>
      </annotation>
    </element>
    <complexType name="TriaxialEllipsoidPropertyType">
      <annotation>
        <documentation>
          gsp:TriaxialEllipsoidPropertyType is a property type for association
          roles to a triaxial ellipsoid,
          either referencing or containing the definition of that ellipsoid.
        </documentation>
      </annotation>
      <sequence minOccurs="0">
        <element ref="gsp:TriaxialEllipsoid"/>
      </sequence>
      <attributeGroup ref="gml:AssociationAttributeGroup"/>
    </complexType>

    <!--
      Datum
    -->
    <element name="CelestialBody" type="gsp:CelestialBodyType" substitutionGroup=
    "gml:Definition">
      <annotation>
        <documentation>
          A gsp:CelestialBody is a name or identifier of the star, planet,
          asteroid or other celestial body
          for which a reference frame is defined.
        </documentation>
      </annotation>
    </element>
    <complexType name="CelestialBodyType">
      <complexContent>
        <extension base="gml:IdentifiedObjectType">
          <!-- We should put a reference to ISO 19112 here, but there is
          apparently no XML schema for it. -->
        </extension>
      </complexContent>
    </complexType>
    <complexType name="CelestialBodyPropertyType">
      <annotation>
        <documentation>
          gsp:CelestialBodyPropertyType is a property type for association roles
          to a celestial body,
          either referencing or containing the definition of that celestial body.
        </documentation>
      </annotation>
      <sequence minOccurs="0">
        <element ref="gsp:CelestialBody"/>
      </sequence>
      <attributeGroup ref="gml:AssociationAttributeGroup"/>
    </complexType>

```

```

    <element name="InertialReferenceFrame" type="gsp:InertialReferenceFrameType"
substitutionGroup="gml:AbstractDatum">
    <annotation>
        <documentation>
            gsp:InertialReferenceFrame is an inertial datum defining the precise
location and orientation
            in 3- or 4-dimensional space of a defined ellipsoid (or sphere), or of
a Cartesian (3D case)
            or Minkowski (4D case) coordinate system centered in this ellipsoid
(or sphere).
        </documentation>
    </annotation>
</element>
<complexType name="InertialReferenceFrameType">
    <complexContent>
        <extension base="gml:AbstractDatumType">
            <sequence>
                <element ref="gml:primeMeridian" minOccurs="0"/>
                <element ref="gml:ellipsoid" minOccurs="0"/>
                <element ref="gsp:celestialBody" minOccurs="0"/>
            <!--
                Departures from gml:GeodeticDatum:
                - Ellipsoid became optional in ISO 19111:2019.
                - Prime meridian is only informative because it rotates with the
planet, so it is okay to omit it.
            -->
            </sequence>
        </extension>
    </complexContent>
</complexType>
<element name="celestialBody" type="gsp:CelestialBodyPropertyType">
    <annotation>
        <documentation>gsp:celestialBody is an association role to the celestial
body used by this inertial datum.</documentation>
    </annotation>
</element>
<complexType name="InertialReferenceFramePropertyType">
    <annotation>
        <documentation>
            gsp:InertialReferenceFramePropertyType is a property type for
association roles to a inertial datum,
            either referencing or containing the definition of that datum.
        </documentation>
    </annotation>
    <sequence minOccurs="0">
        <element ref="gsp:InertialReferenceFrame"/>
    </sequence>
    <attributeGroup ref="gml:AssociationAttributeGroup"/>
</complexType>

<!--
Coordinate Reference System
-->
<element name="InertialCRS" type="gsp:InertialCRSType" substitutionGroup=
"gml:AbstractSingleCRS"/>
<complexType name="InertialCRSType">
    <annotation>
        <documentation>gsp:InertialCRS is a coordinate reference system with
axes having fixed directions relative to stars.</documentation>
    </annotation>
    <complexContent>

```

```

    <extension base="gml:AbstractCRSType">
      <sequence>
        <choice>
          <element ref="gml:ellipsoidalCS"/>
          <element ref="gml:cartesianCS"/>
          <element ref="gml:sphericalCS"/>
          <element ref="gsp:minkowskiCS"/>
        </choice>
        <element ref="gsp:inertialReferenceFrame"/>
      </sequence>
    </extension>
  </complexContent>
</complexType>
<element name="minkowskiCS" type="gsp:MinkowskiCSPropertyType">
  <annotation>
    <documentation>gsp:minkowskiCS is an association role to the Minkowski
coordinate system used by this CRS.</documentation>
  </annotation>
</element>
<element name="inertialReferenceFrame" type="gsp:InertialReferenceFrameProper
tyType">
  <annotation>
    <documentation>gsp:inertialReferenceFrame is an association role to the
inertial datum used by this CRS.</documentation>
  </annotation>
</element>
<complexType name="InertialCRSPropertyType">
  <annotation>
    <documentation>gsp:InertialCRSPropertyType is a property type
for association roles to a inertial coordinate reference system, either
referencing or containing the definition of that reference system.</
documentation>
  </annotation>
  <sequence minOccurs="0">
    <element ref="gsp:InertialCRS"/>
  </sequence>
  <attributeGroup ref="gml:AssociationAttributeGroup"/>
</complexType>
</schema>

```

Figure B.1 — XML schema proposal for standardization by a SWG

B.1. Examples

This Testbed provides two example files compliant with above XSD schema: `Didymos.xml` and `Dimorphos.xml` in the examples sub-directory. The context for those examples is presented in Clause 2.4. Below is the Dimorphos example:

```

<?xml version="1.0" encoding="UTF-8"?>
<gsp:InertialCRS xsi:schemaLocation = "link:++http://www.opengis.net/gsp/1.0+
+[] ../geospatialInSpace.xsd"
  xmlns:gsp = "link:++http://www.opengis.net/gsp/1.0"+
+[]
  xmlns:gml = "link:++http://www.opengis.net/gml/3.2"+
+[]
  xmlns:xsi = "link:++http://www.w3.org/2001/XMLSchema-
instance"++[]
  xmlns:xlink = "link:++http://www.w3.org/1999/xlink"++[]

```

```

        gml:id = "DimorphosCRS">
      <gml:identifier codeSpace="JPL:HORIZONS">urn:ogc:def:crs:JPL::120065803</gml:
identifier>
      <gml:name codeSpace="JPL:HORIZONS">Dimorphos satellite (inertial)</gml:name>
      <gml:scope>For use with the reconstructed trajectory of the Didymos system.</
gml:scope>
      <gml:cartesianCS>
        <gml:CartesianCS gml:id="DimorphosCS">
          <gml:identifier codeSpace="JPL:HORIZONS">urn:ogc:def:cs:JPL::ICRF</gml:
identifier>
          <gml:name>Cartesian 3D CS (km).</gml:name>
          <gml:axis>
            <gml:CoordinateSystemAxis gml:id="axisX" uom="urn:ogc:def:uom:EPSG:
:9036">
              <gml:identifier codeSpace="JPL:HORIZONS">urn:ogc:def:axis:JPL::ICRF-X
</gml:identifier>
              <gml:name>X</gml:name>
              <gml:axisAbbrev>X</gml:axisAbbrev>
              <gml:axisDirection codeSpace="JPL:HORIZONS">X</gml:axisDirection>
            </gml:CoordinateSystemAxis>
          </gml:axis>
          <gml:axis>
            <gml:CoordinateSystemAxis gml:id="axisY" uom="urn:ogc:def:uom:EPSG:
:9036">
              <gml:identifier codeSpace="JPL:HORIZONS">urn:ogc:def:axis:JPL::ICRF-Y
</gml:identifier>
              <gml:name>Y</gml:name>
              <gml:axisAbbrev>Y</gml:axisAbbrev>
              <gml:axisDirection codeSpace="JPL:HORIZONS">Y</gml:axisDirection>
            </gml:CoordinateSystemAxis>
          </gml:axis>
          <gml:axis>
            <gml:CoordinateSystemAxis gml:id="axisZ" uom="urn:ogc:def:uom:EPSG:
:9036">
              <gml:identifier codeSpace="JPL:HORIZONS">urn:ogc:def:axis:JPL::ICRF-Z
</gml:identifier>
              <gml:name>Z</gml:name>
              <gml:axisAbbrev>Z</gml:axisAbbrev>
              <gml:axisDirection codeSpace="JPL:HORIZONS">Z</gml:axisDirection>
            </gml:CoordinateSystemAxis>
          </gml:axis>
        </gml:CartesianCS>
      </gml:cartesianCS>
      <gsp:inertialReferenceFrame>
        <gsp:InertialReferenceFrame gml:id="DimorphosDatum">
          <gml:identifier codeSpace="JPL:HORIZONS">urn:ogc:def:datum:JPL::120065803
</gml:identifier>
          <gml:name>Dimorphos inertial reference frame</gml:name>
          <gml:scope>For orbit positioning.</gml:scope>
          <gsp:celestialBody>
            <gsp:CelestialBody gml:id="Dimorphos">
              <gml:identifier codeSpace="JPL:HORIZONS">urn:ogc:def:body:JPL:
:120065803</gml:identifier>
              <gml:name>Dimorphos</gml:name>
            </gsp:CelestialBody>
          </gsp:celestialBody>
          <gml:ellipsoid>
            <gsp:TriaxialEllipsoid gml:id="DimorphosEllipsoid">
              <gml:identifier codeSpace="JPL:HORIZONS">urn:ogc:def:ellipsoid:JPL:
:120065803</gml:identifier>
              <gml:name>Dimorphos ellipsoid</gml:name>
            </gsp:TriaxialEllipsoid>
          </gml:ellipsoid>
        </gsp:InertialReferenceFrame>
      </gsp:inertialReferenceFrame>
    </gml:CartesianCS>
  </gml:cartesianCS>
</gml:CRS>

```

```

        <gml:semiMajorAxis uom="urn:ogc:def:uom:EPSG::9001">88.5</gml:
semiMajorAxis>
        <gml:secondDefiningParameter>
        <gml:SecondDefiningParameter>
        <gml:semiMinorAxis uom="urn:ogc:def:uom:EPSG::9001">58</gml:
semiMinorAxis>
        </gml:SecondDefiningParameter>
        </gml:secondDefiningParameter>
        <gsp:semiMedianAxis uom="urn:ogc:def:uom:EPSG::9001">87</gsp:
semiMedianAxis>
        </gsp:TriaxialEllipsoid>
        </gml:ellipsoid>
        <!-- Prime meridian is optional for an inertial reference frame. -->
        </gsp:InertialReferenceFrame>
        </gsp:inertialReferenceFrame>
</gsp:InertialCRS>

```

Figure B.2 — Definition of Didymos inertial CRS in extended GML



ANNEX C (INFORMATIVE) GEOAPI INTERFACES



ANNEX C

(INFORMATIVE)

GEOAPI INTERFACES

This annex lists the new GeoAPI interfaces introduced in this Testbed. Those interfaces have been derived from the abstract model described in Clause 2. The snippets shown below omit the license headers, some Javadoc tags, package and import statements for brevity, but are available in their full version in a [branch on GitHub](#). **TODO: need a more permanent location for that branch.** Those snippets are built on top of the following existing interfaces:

- CoordinateSystem, Datum, Ellipsoid, IdentifiedObject, PrimeMeridian and SingleCRS are GeoAPI 3.0 interfaces derived from ISO 19111.
- Location is an interface derived from ISO 19112 and currently in GeoAPI-pending (not yet released as a standard).

```
/**
 * Identifies the star, planet, asteroid or other celestial body for which a
 * reference frame is defined.
 */
public interface CelestialBody extends IdentifiedObject {
    /**
     * Returns the location of this body in reference system by identifiers.
     *
     * @see ReferenceSystemUsingIdentifiers
     */
    default Optional<Location> getLocation() {
        return Optional.empty();
    }
}
```

Figure C.1 — Celestial Body interface proposal for the GeoAPI Standard (Java version)

In the Datum parent interface (not shown here), an optional `getCelestialBody()` method is added. Then:

```
/**
 * Origin and orientation of an inertial CRS.
 * An inertial reference frame defines the precise location and orientation in
 * 3- or 4-dimensional space
 * of a defined ellipsoid (or sphere), or of a Cartesian (3D case) or
 * Minkowski (4D case) coordinate system
 * centered in this ellipsoid (or sphere).
 */
public interface InertialReferenceFrame extends Datum {
    /**
     * Returns the ellipsoid.
     */
    default Optional<Ellipsoid> getEllipsoid() {
```

```

        return Optional.empty();
    }

    /**
     * Returns the prime meridian.
     */
    default Optional<PrimeMeridian> getPrimeMeridian() {
        return Optional.empty();
    }
}

```

Figure C.2 — Inertial Reference Frame interface proposal for the GeoAPI Standard (Java version)

Used by:

```

/**
 * A 2-, 3- or 4-dimensional coordinate reference system with axes at fixed
 * position relative to stars.
 *
 * <p>This type of CRS can be used with coordinate systems of type
 * {@link org.opengis.referencing.cs.EllipsoidalCS},
 * {@link org.opengis.referencing.cs.SphericalCS},
 * {@link org.opengis.referencing.cs.CartesianCS} or
 * {@link org.opengis.referencing.cs.MinkowskiCS}.</p>
 */
public interface InertialCRS extends SingleCRS {
    /**
     * Returns the datum, which shall be inertial.
     */
    @Override
    InertialReferenceFrame getDatum();
}

```

Figure C.3 — Inertial CRS interface proposal for the GeoAPI Standard (Java version)

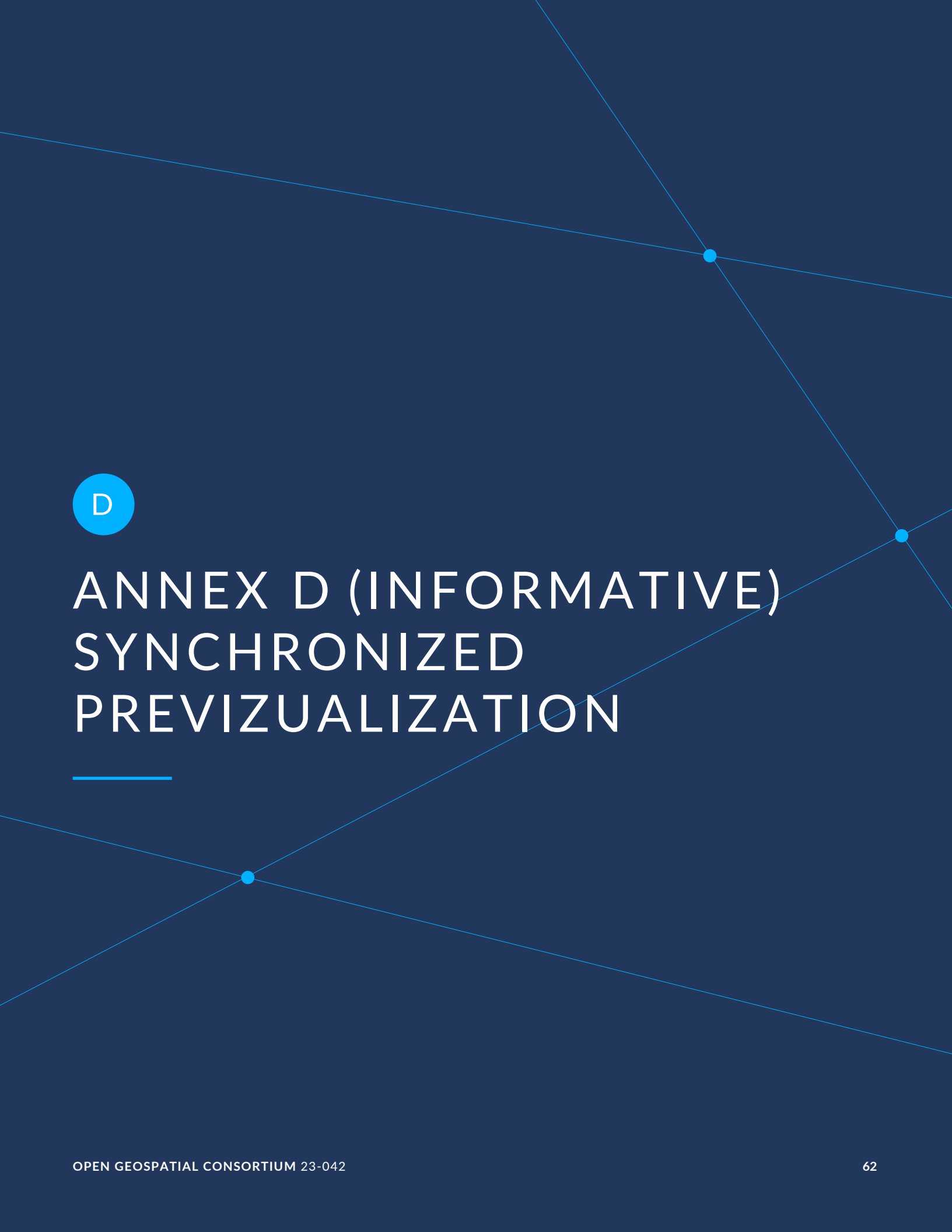
May use:

```

/**
 * A 4-dimensional spatio-temporal coordinate system with time expressed as
 * lengths.
 * The three spacial dimensions give the position of points relative to
 * orthogonal straight axes.
 * The fourth dimension gives the position in time since an epoch.
 * All axes shall have the same length unit of measure,
 * with time expressed as the distance covered by light in the vacuum during
 * the elapsed time.
 *
 * <p>This type of CS can be used by coordinate reference systems of type
 * {@link org.opengis.referencing.crs.InertialCRS} or
 * {@link org.opengis.referencing.crs.EngineeringCRS}.</p>
 */
public interface MinkowskiCS extends CoordinateSystem {
}

```

Figure C.4 — Minkowski CS interface proposal for the GeoAPI Standard (Java version)



D

ANNEX D (INFORMATIVE) SYNCHRONIZED PREVIZUALIZATION



ANNEX D

(INFORMATIVE)

SYNCHRONIZED PREVIZUALIZATION

These screenshots show data synchronized using WebVMT in a web browser. The trajectory of the StreetDrone recorded on board is shown in orange, synchronized with its trajectory observed by the static cameras in blue. A crosshair marks the frame centre and this line of sight is shown on the map as a faint blue line with the camera marked as a blue dot.

Three examples are included in this report to demonstrate the accuracy of the synchronization process.

D.1. Example 17:15-17:21, 25th April 2023



Figure D.1 – Synchronized Previzualization - Moto G30, 17:15-17:21 25 April, Pass 1



Figure D.2 — Synchronized Previzualization - Moto G30, 17:15-17:21 25 April, Pass 2



Figure D.3 — Synchronized Previzualization - Moto G30, 17:15-17:21 25 April, Pass 3



Figure D.4 — Synchronized Previzualization - Moto G30, 17:15-17:21 25 April, Pass 4



Figure D.5 — Synchronized Previzualization - Nokia 5, 17:15-17:21 25 April, Pass 1

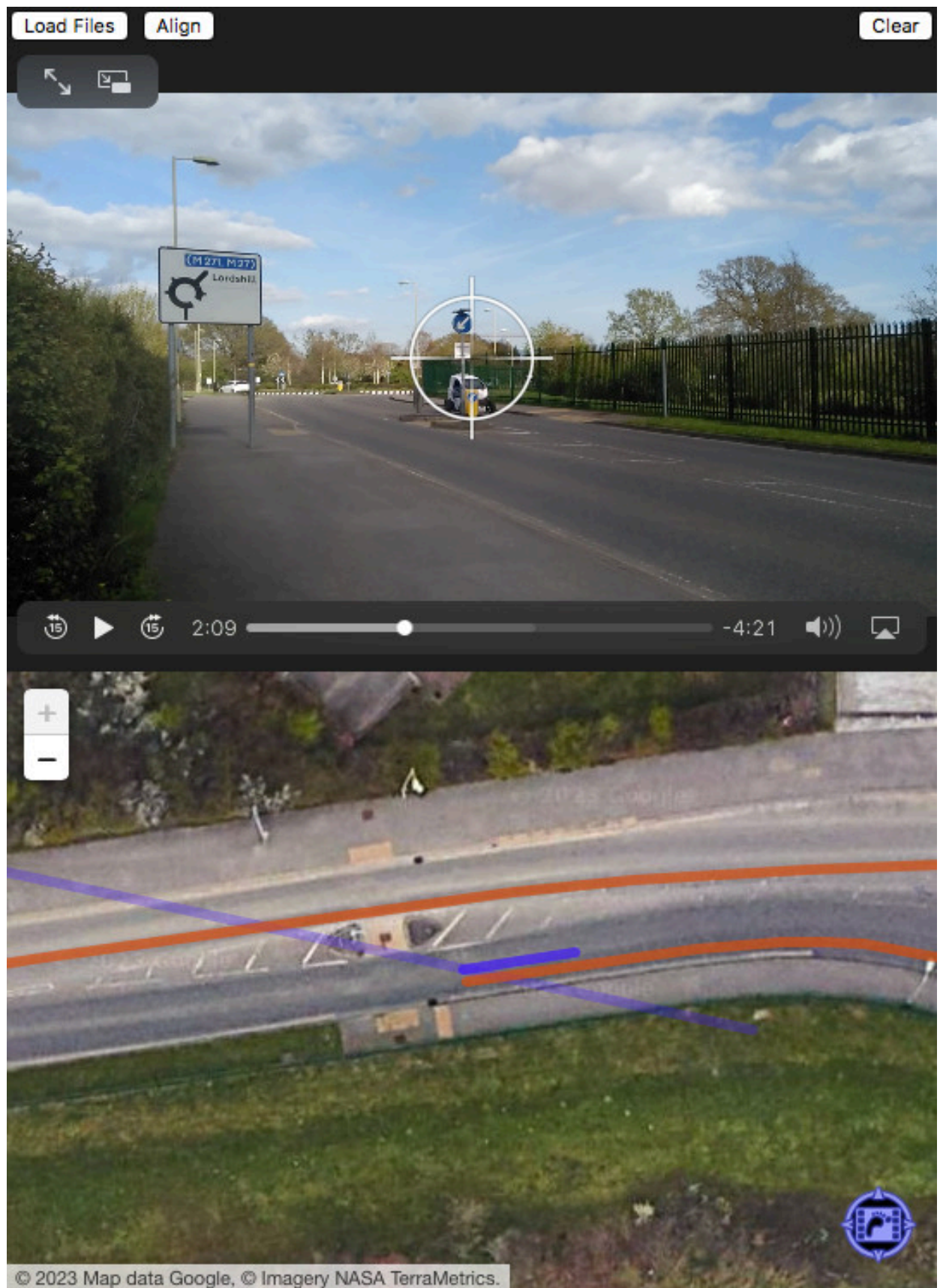


Figure D.6 — Synchronized Previzualization - Nokia 5, 17:15-17:21 25 April, Pass 2

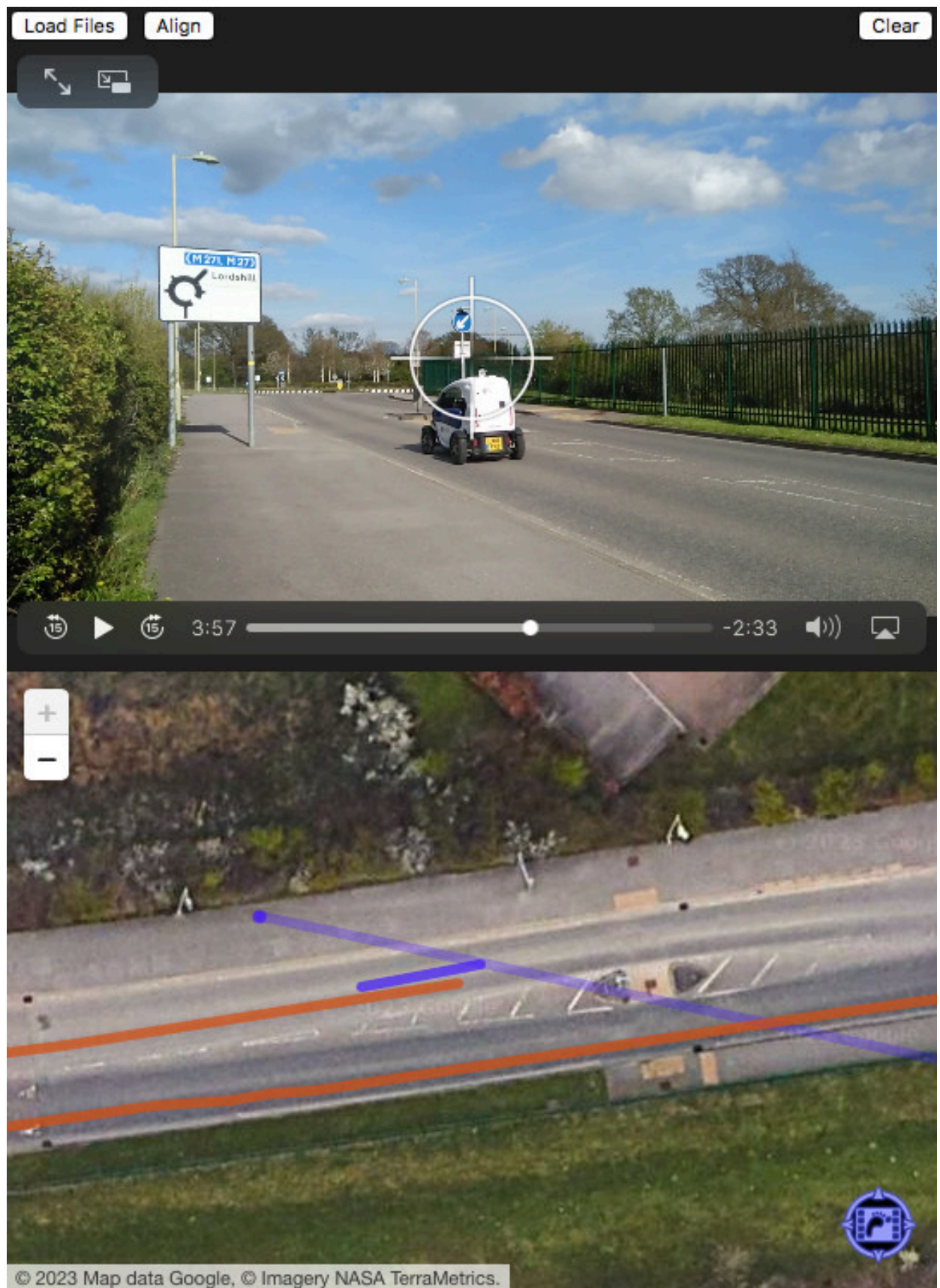


Figure D.7 — Synchronized Previzualization - Nokia 5, 17:15-17:21 25 April, Pass 3



Figure D.8 — Synchronized Previzualization - Nokia 5, 17:15-17:21 25 April, Pass 4

D.2. Example 12:02-12:09, 26th April 2023

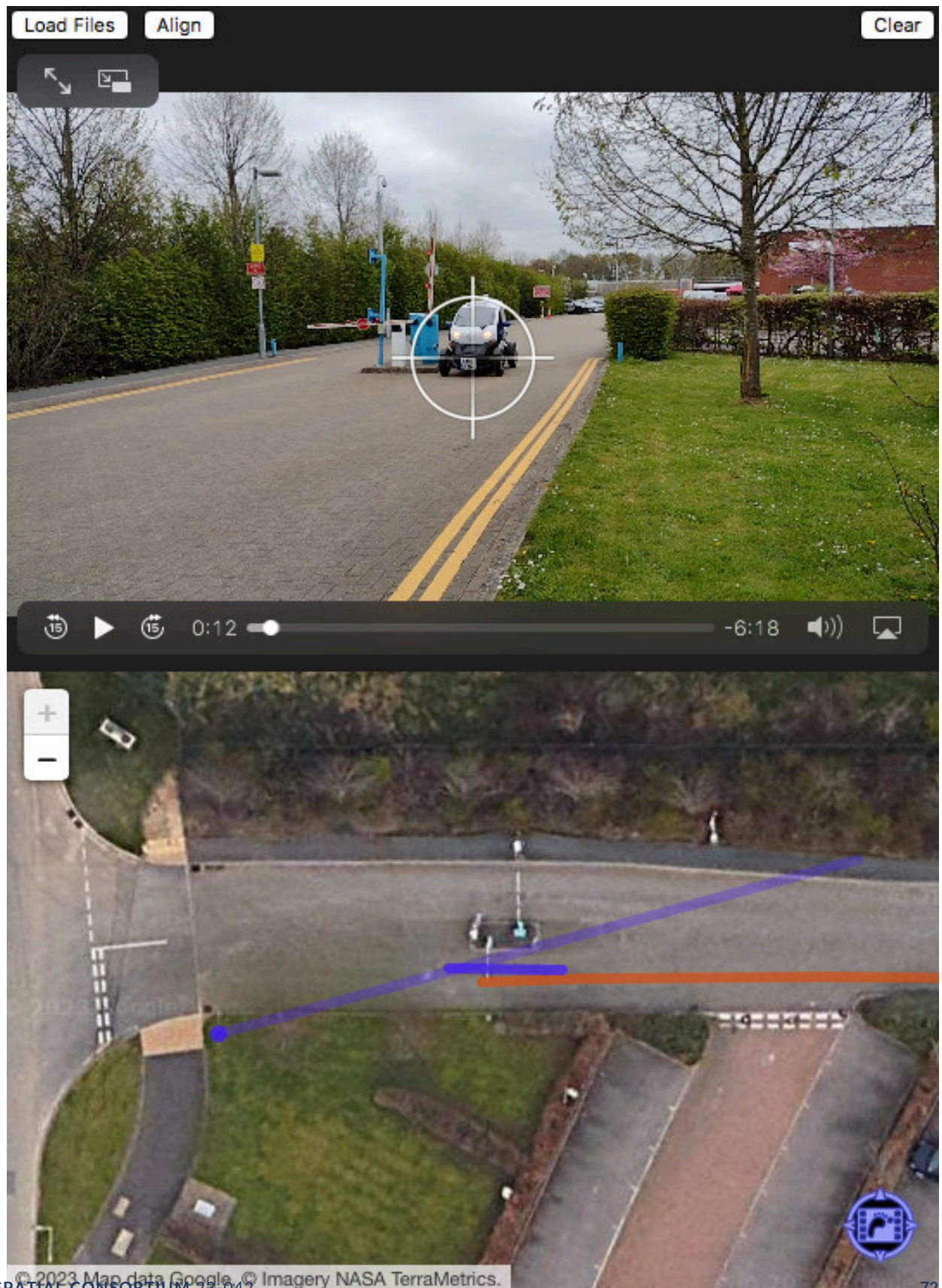


Figure D.9 – Synchronized Previzualization - Moto G30, 12:02-12:09 26 April, Pass 1



Figure D.10 — Synchronized Previzualization - Moto G30, 12:02-12:09 26 April, Pass 2

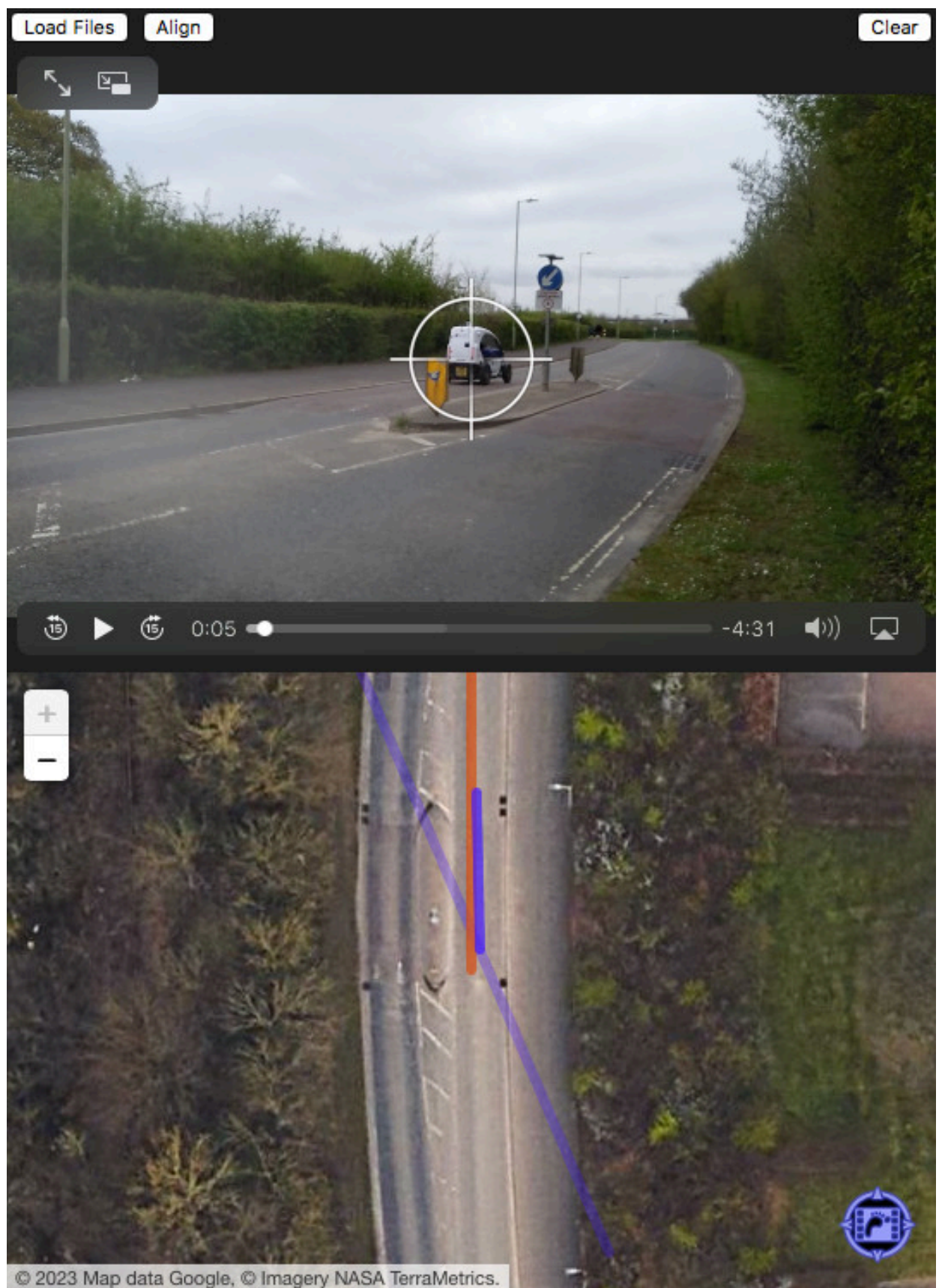


Figure D.11 — Synchronized Previzualization - Nokia 5, 12:02-12:09 26 April, Pass 1

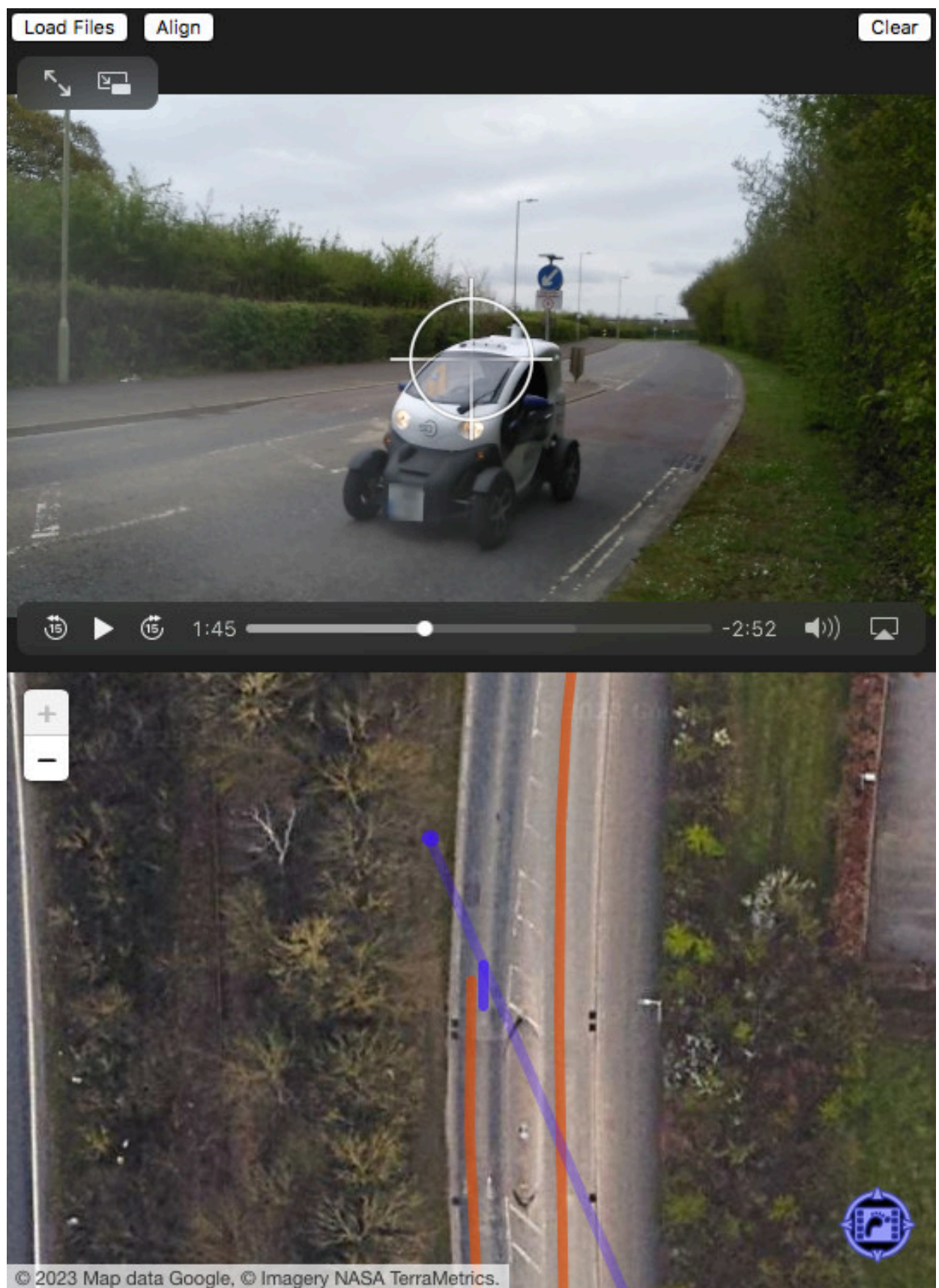


Figure D.12 — Synchronized Previzualization - Nokia 5, 12:02-12:09 26 April, Pass 2



Figure D.13 — Synchronized Previzualization - Nokia 5, 12:02-12:09 26 April, Pass 3

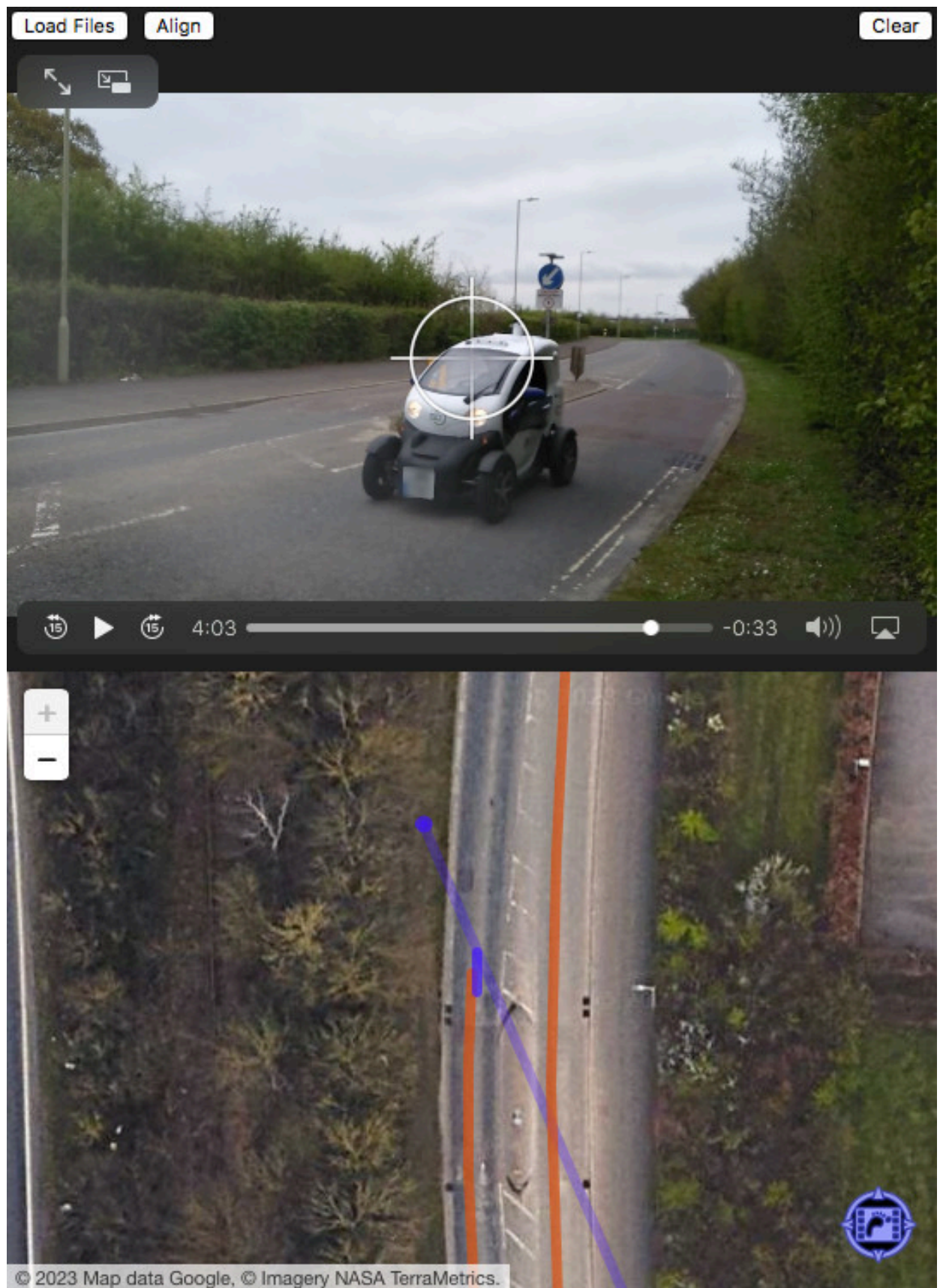


Figure D.14 — Synchronized Previzualization - Nokia 5, 12:02-12:09 26 April, Pass 4

D.3. Example 13:08-13:14, 26th April 2023



Figure D.15 — Synchronized Previzualization - Moto G30, 13:08-13:14 26 April, Pass 1



Figure D.16 — Synchronized Previzualization - Moto G30, 13:08-13:14 26 April, Pass 2



Figure D.17 — Synchronized Previzualization - Moto G30, 13:08-13:14 26 April, Pass 3

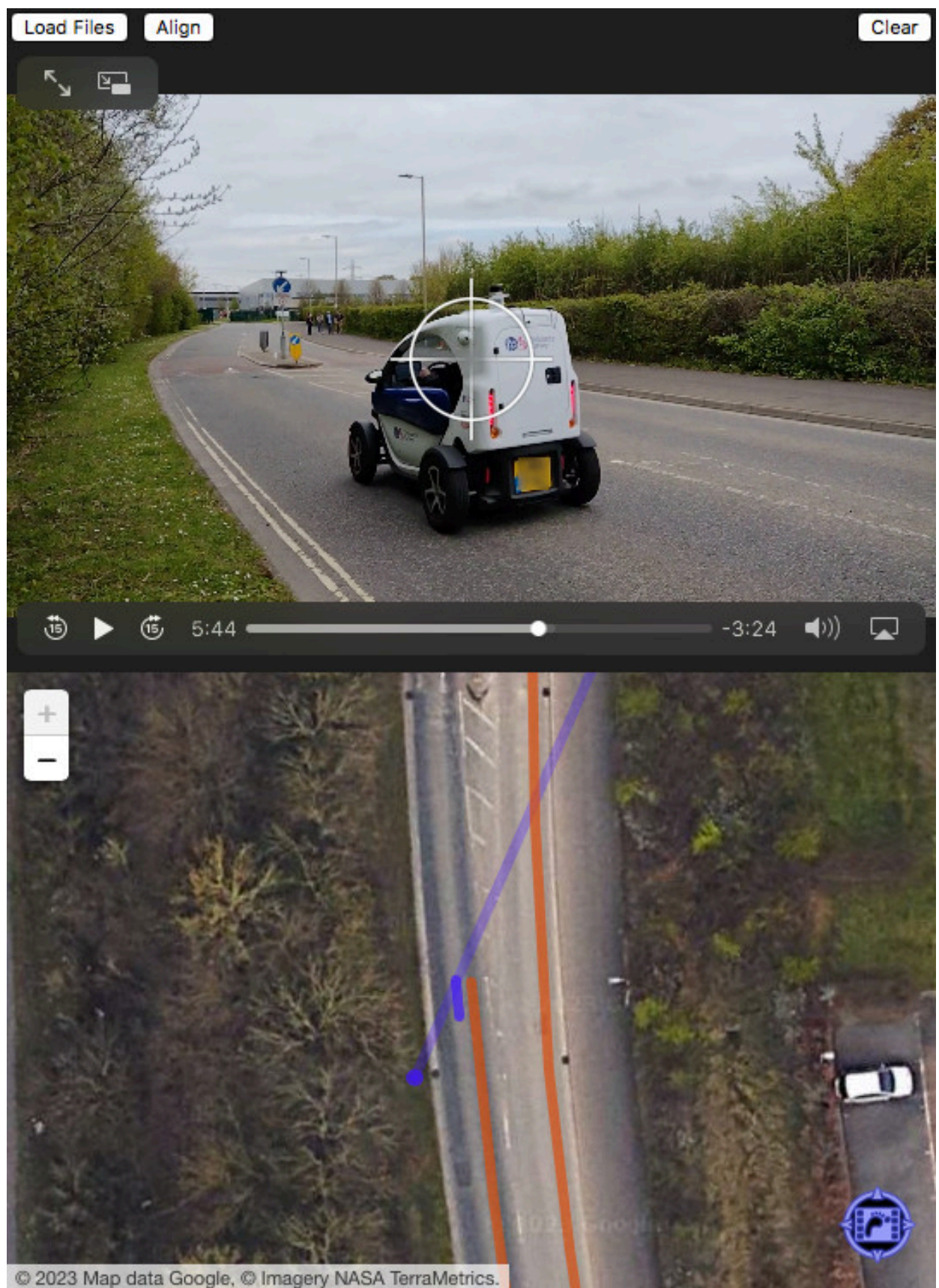


Figure D.18 — Synchronized Previzualization - Moto G30, 13:08-13:14 26 April, Pass 4

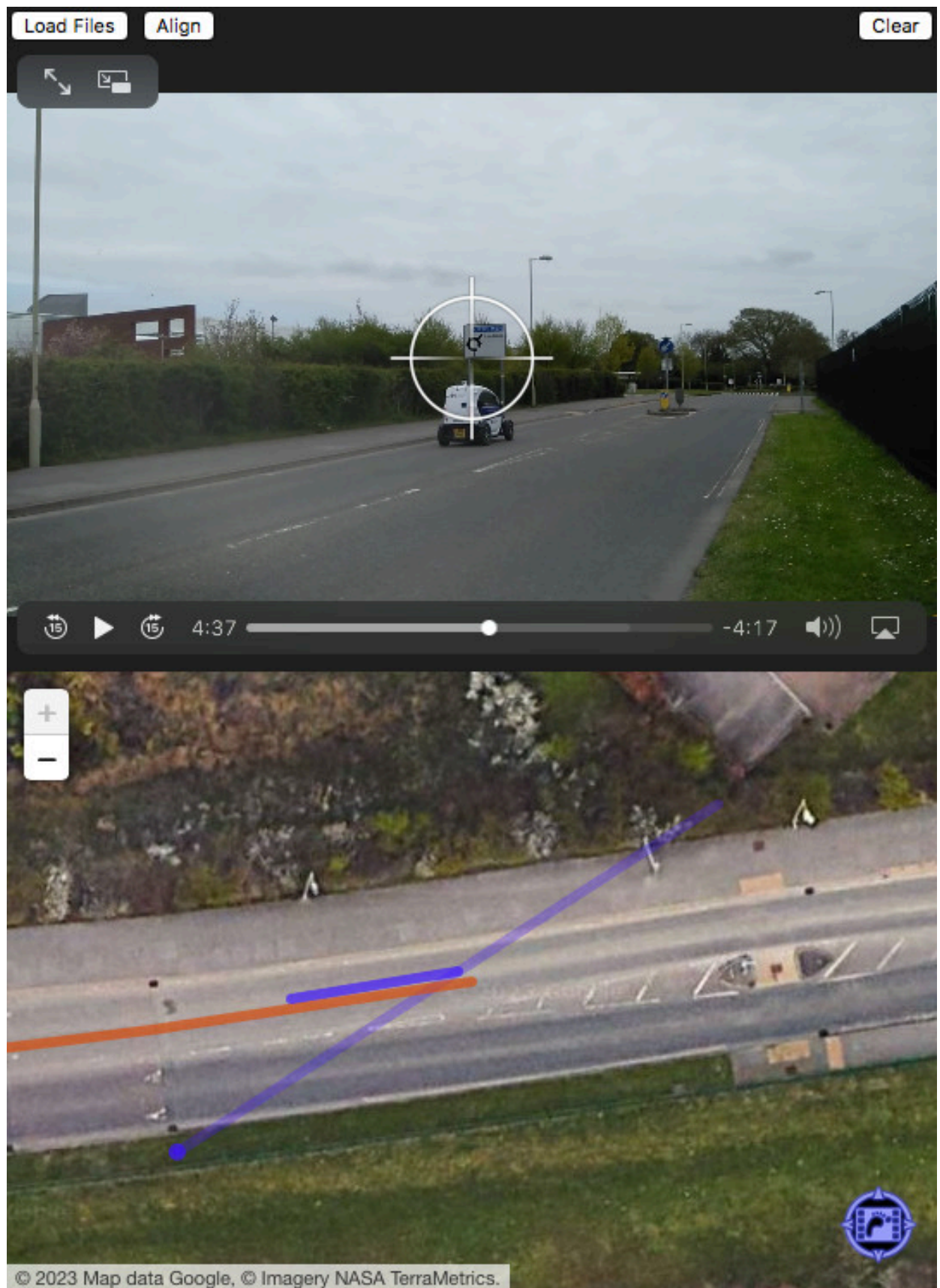


Figure D.19 — Synchronized Previzualization - Nokia 5, 13:08-13:14 26 April, Pass 1

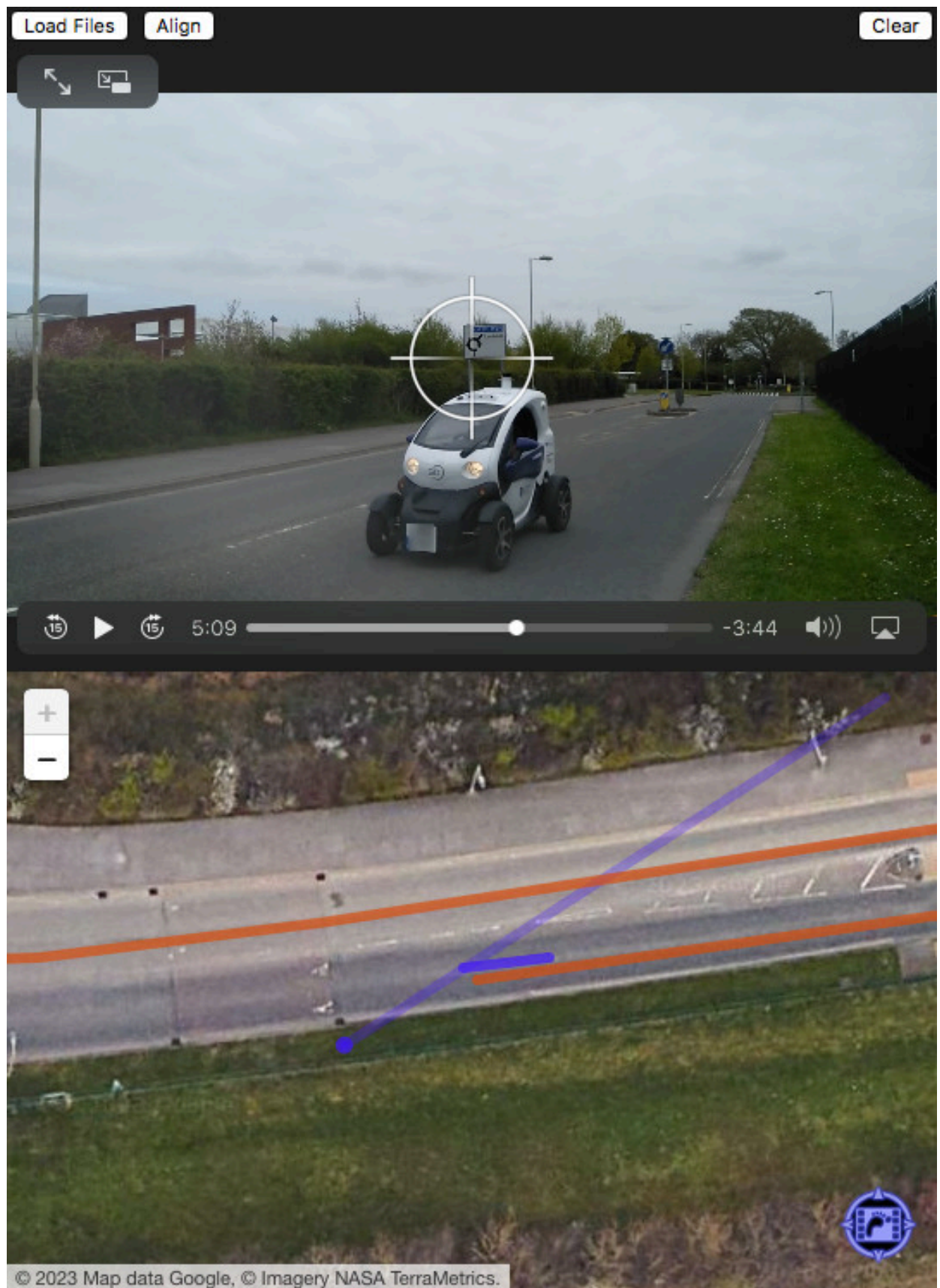


Figure D.20 — Synchronized Previzualization - Nokia 5, 13:08-13:14 26 April, Pass 2

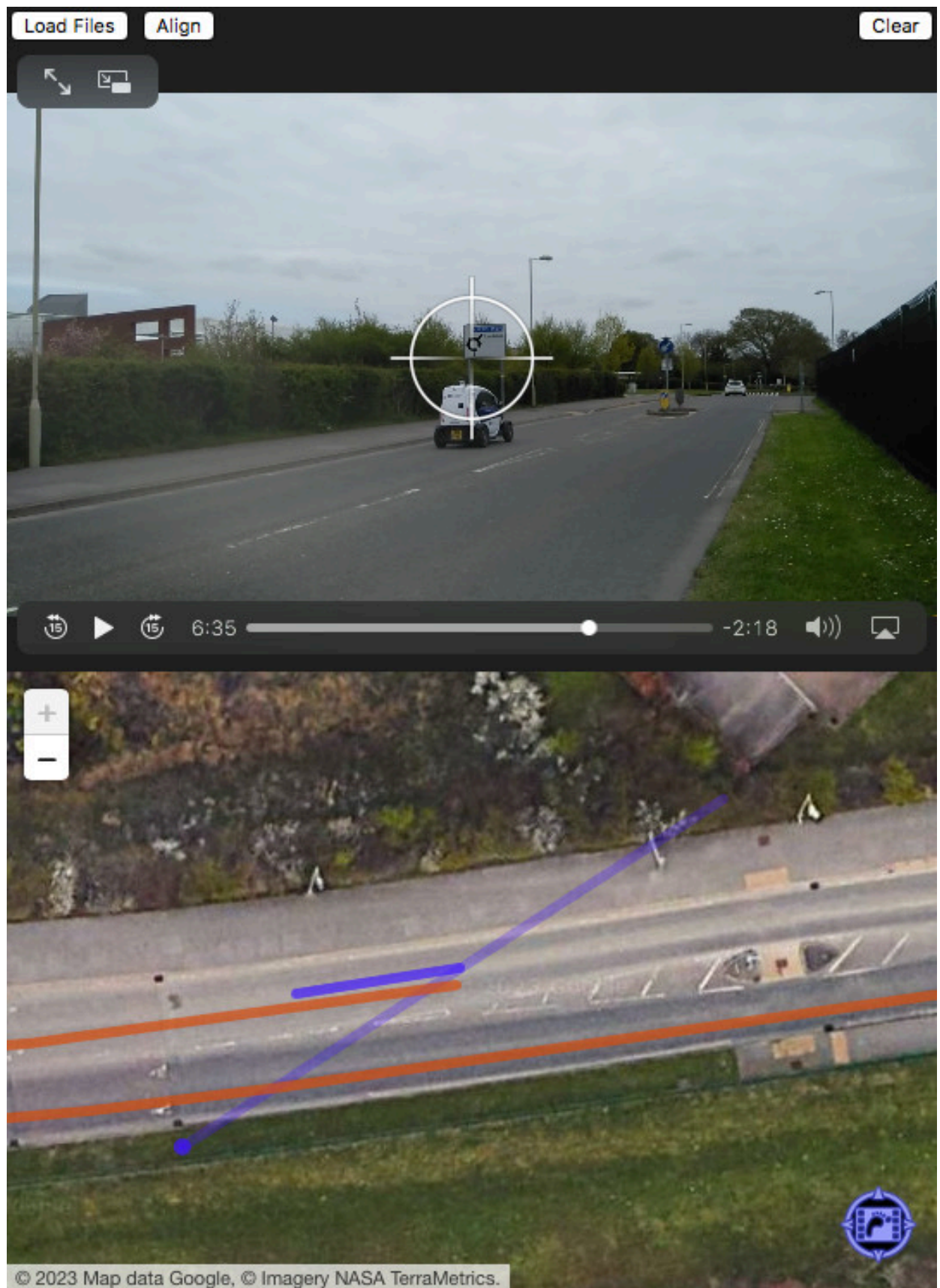


Figure D.21 — Synchronized Previzualization - Nokia 5, 13:08-13:14 26 April, Pass 3

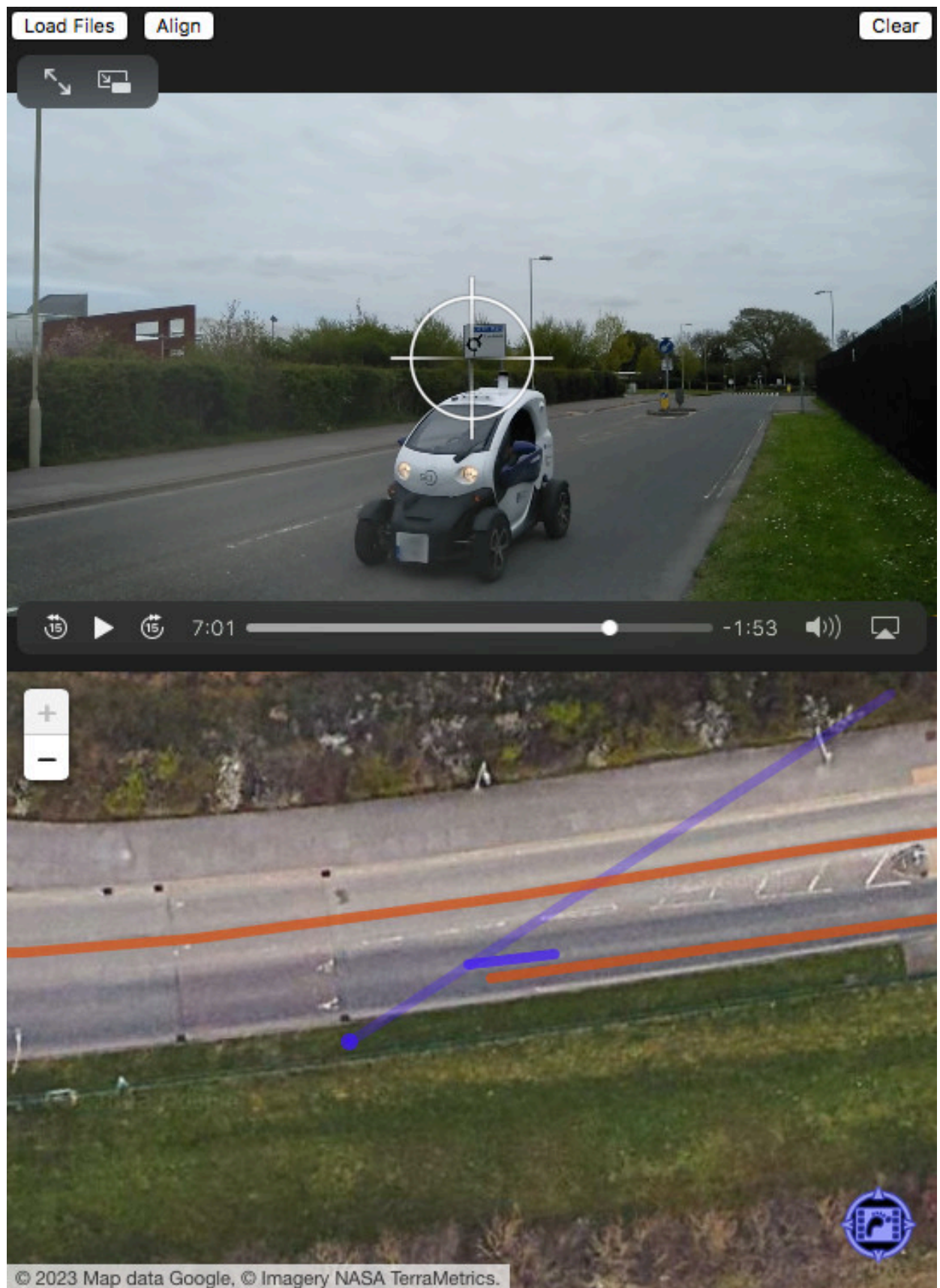


Figure D.22 — Synchronized Previzualization - Nokia 5, 13:08-13:14 26 April, Pass 4



BIBLIOGRAPHY





BIBLIOGRAPHY

- [1] Roger Lott: OGC 18-005r5, *Topic 2 – Referencing by coordinates Corrigendum*. Open Geospatial Consortium (2021). <https://docs.ogc.org/as/18-005r5/18-005r5.html>.
- [2] Martin Daly: OGC 01-009, *Coordinate Transformation Services – OLE/COM*. Open Geospatial Consortium (2001). https://portal.ogc.org/files/?artifact_id=999.
- [3] Martin Desruisseaux: OGC 22-038r2, *Testbed-18: Reference Frame Transformation Engineering Report*. Open Geospatial Consortium (2023). <https://docs.ogc.org/per/22-038r2.html>.
- [4] Akinori Asahara, Ryosuke Shibasaki, Nobuhiro Ishimaru, David Burggraf: OGC 14-084r2, *OGC® Moving Features Encoding Extension: Simple Comma Separated Values (CSV)*. Open Geospatial Consortium (2015). <https://docs.ogc.org/is/14-084r2/14-084r2.html>.
- [5] IOGP: *EPSG Geodetic Parameter Dataset*. <https://epsg.org>
- [6] Karney, C.F.F: Algorithms for geodesics. *J Geod* 87, 43–55 (2013). <https://doi.org/10.1007/s00190-012-0578-z>